# The Systems Behind Modern AI

## Why and How Hardware Designed for 3D Graphics Powers Modern AI

Sam Ly

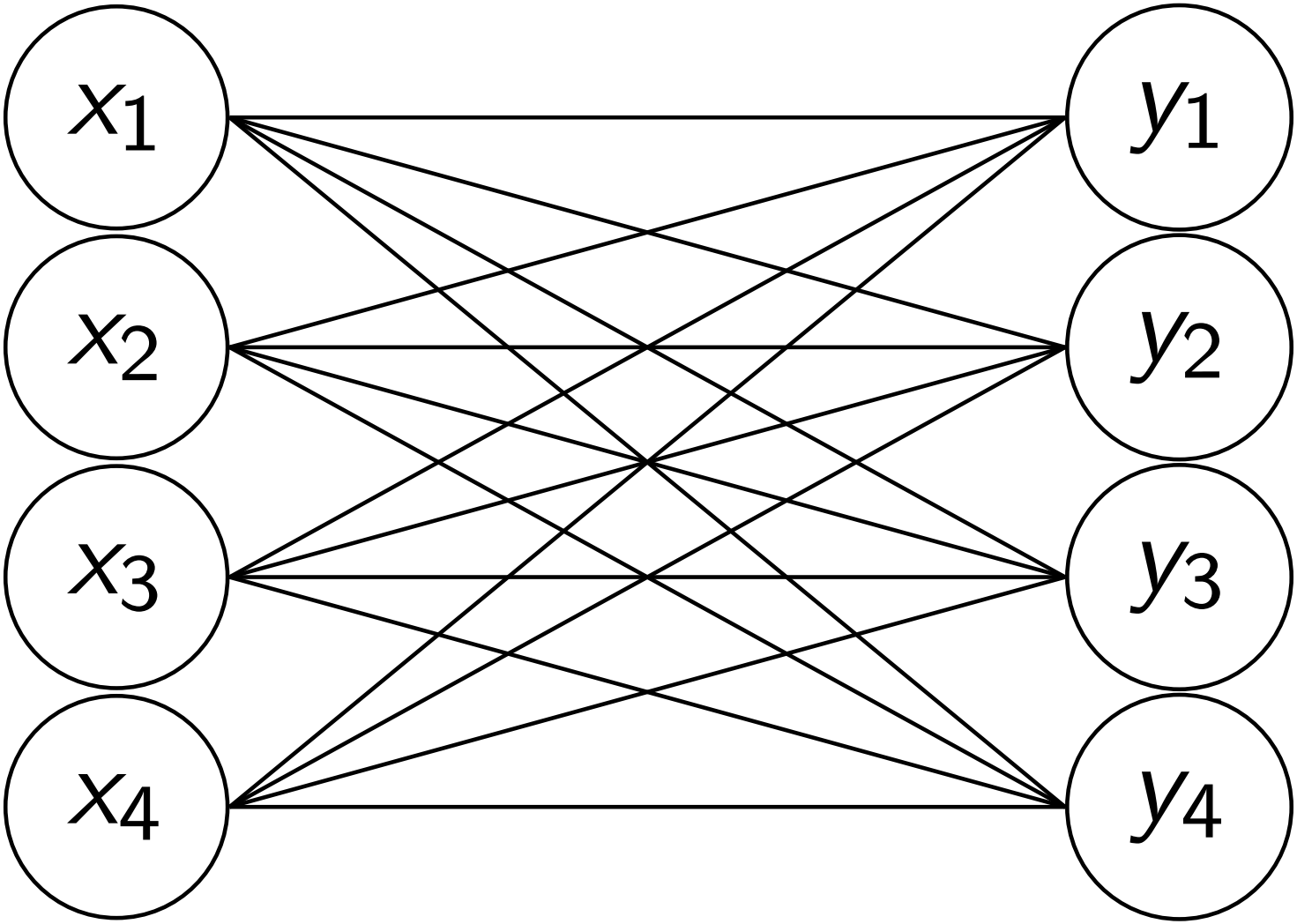
Cal Poly Pomona

Fall 2026

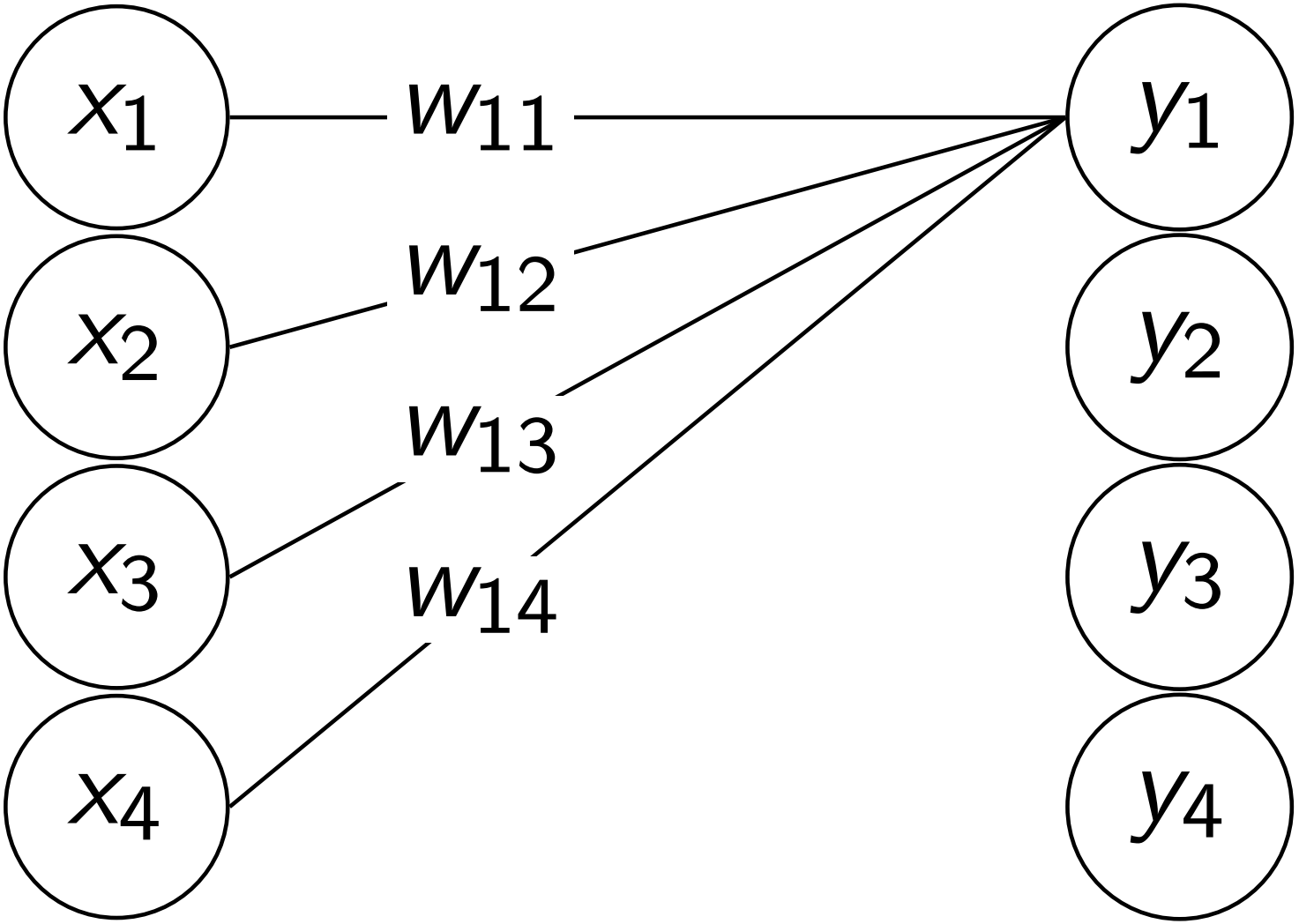
# Table of Contents


1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

# Basic Neural Net (MLP/FFN)

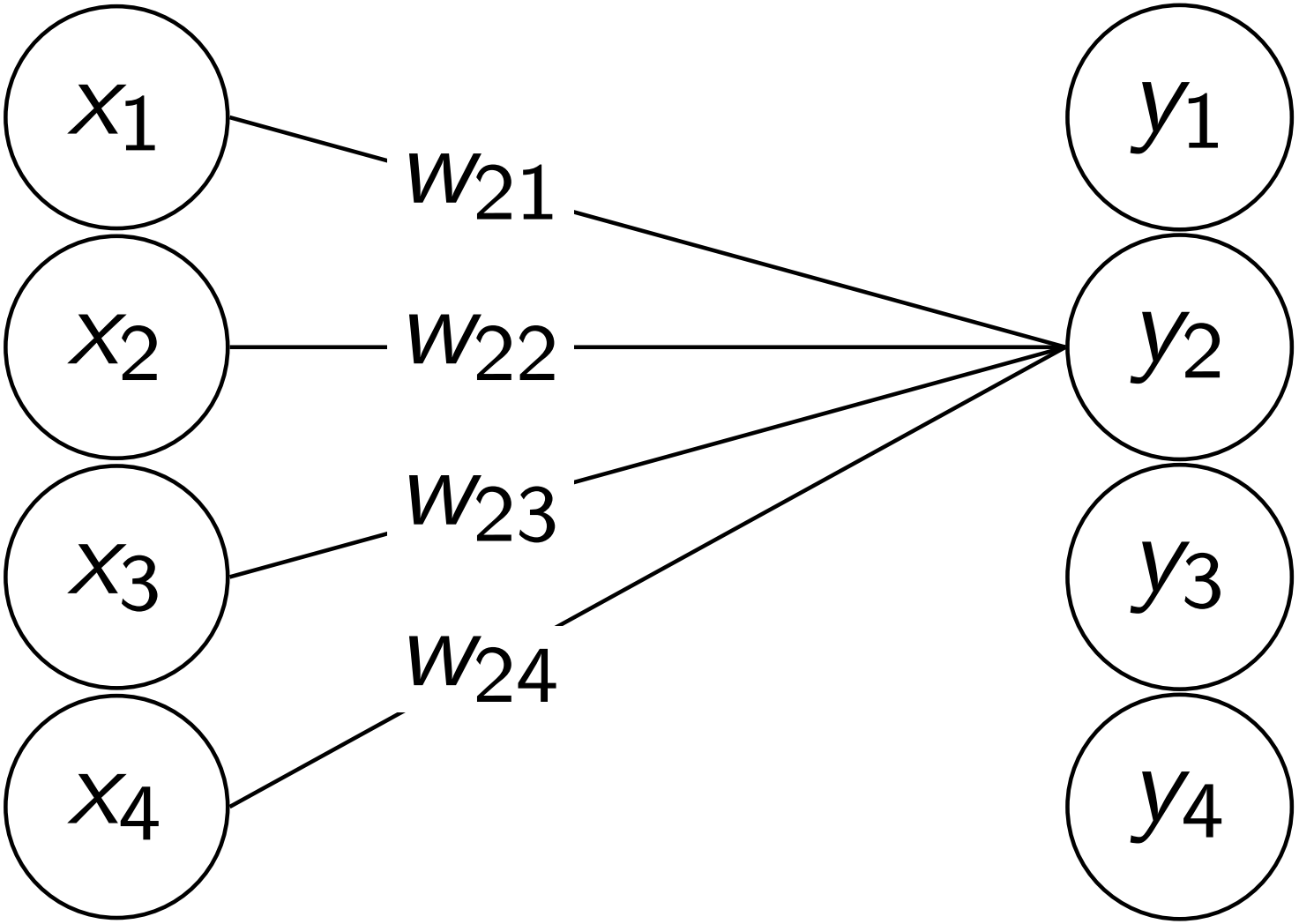


# Basic Neural Net (MLP/FFN)



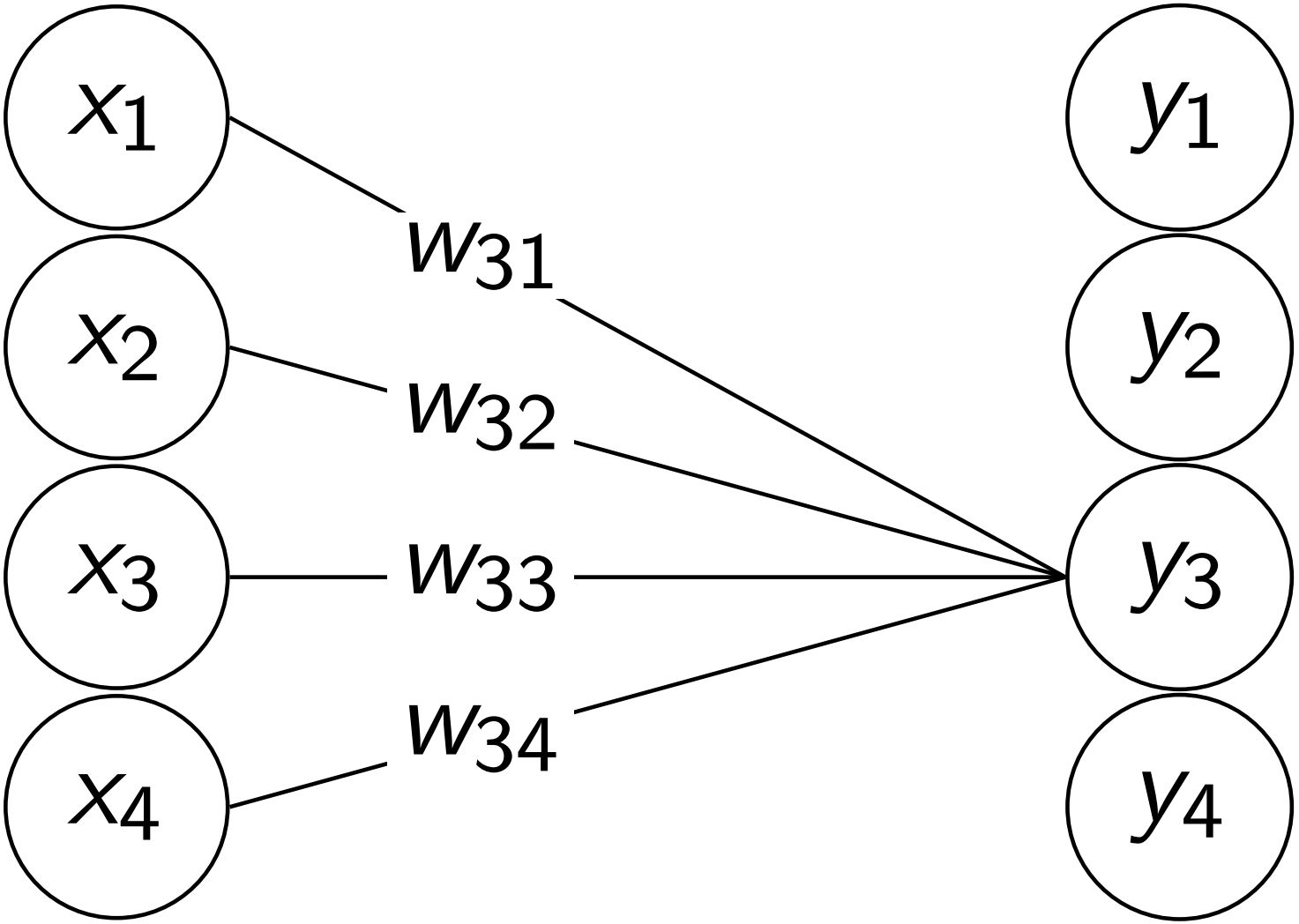

$$y_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14}$$

# Basic Neural Net (MLP/FFN)



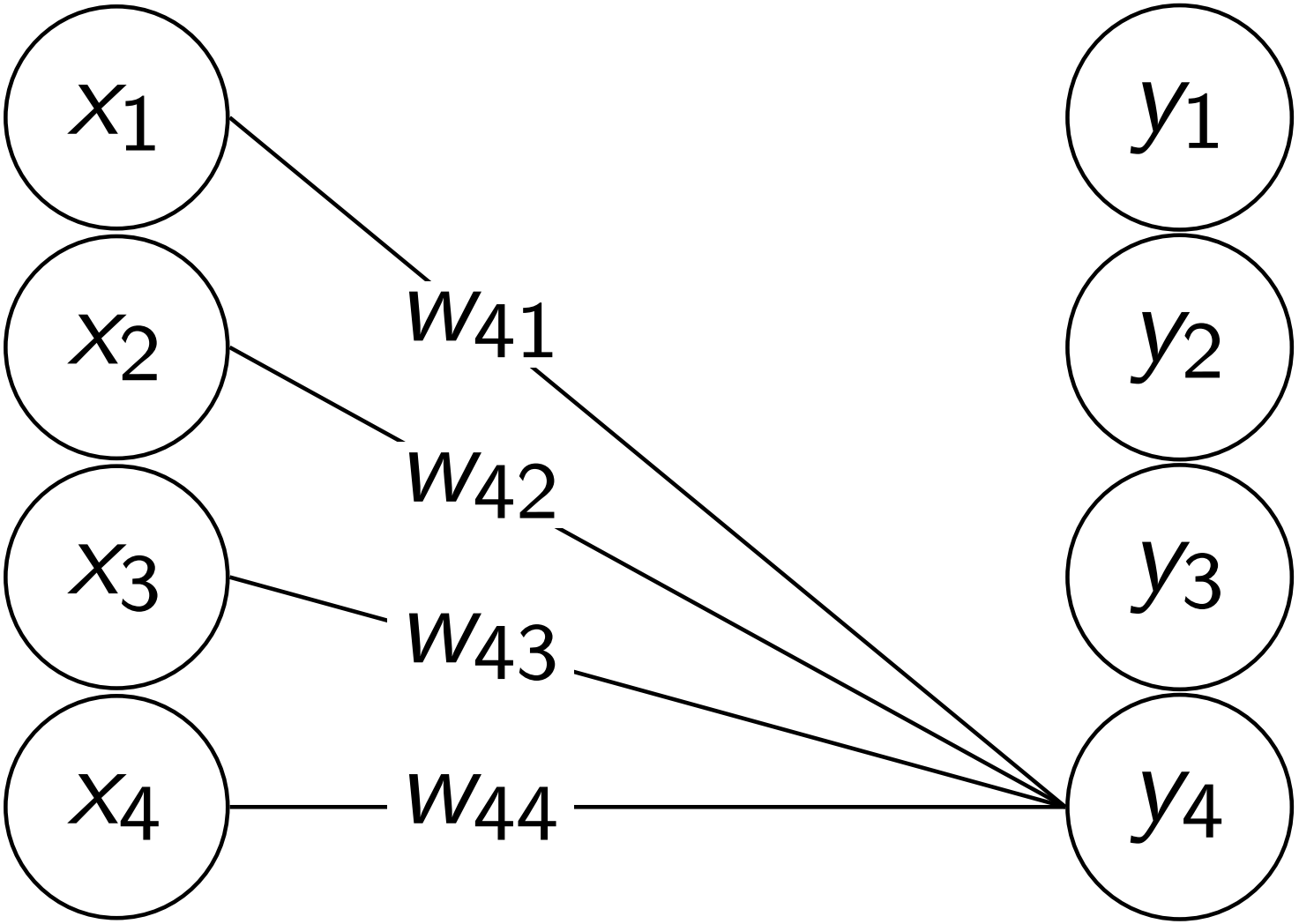

$$y_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24}$$

# Basic Neural Net (MLP/FFN)




$$y_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34}$$

# Basic Neural Net (MLP/FFN)




$$y_4 = x_1 w_{41} + x_2 w_{42} + x_3 w_{43} + x_4 w_{44}$$

# Linear Algebra Interpretation

Here, notice that for each $y_i$, we are taking a weighted sum of the $x$ elements. That is,

$$y_i = x_1 w_{i1} + x_2 w_{i2} + x_3 w_{i3} + x_4 w_{i4}.$$

# Linear Algebra Interpretation

Here, notice that for each $y_i$, we are taking a weighted sum of the $x$ elements. That is,

$$y_i = x_1 w_{i1} + x_2 w_{i2} + x_3 w_{i3} + x_4 w_{i4}.$$

Arranging the $y_i$ into a vector, we get

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \\ w_{41} & w_{42} & w_{43} & w_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}.$$

# Linear Algebra Interpretation

We can generalize the number of $x$'s and the number of $y$'s.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

# Linear Algebra Interpretation

We can generalize the number of $x$'s and the number of $y$'s.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

We say, $n$ is the *input dimension* and $m$ is the *output dimension*.

# Computing with multiple $x$ vectors.

Suppose we have multiple $x$ vectors, for which we want to pass through our neural network. We can stack the multiple $x$ vectors in the **batch**, which we compute with a matrix multiplication.

# Computing with multiple $x$ vectors.

Suppose we have multiple $x$ vectors, for which we want to pass through our neural network. We can stack the multiple $x$ vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

# Computing with multiple $x$ vectors.

Suppose we have multiple $x$ vectors, for which we want to pass through our neural network. We can stack the multiple $x$ vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

We write this succinctly as $Y = WX$, where $Y \in \mathbb{R}^{m \times B}$, $X \in \mathbb{R}^{n \times B}$, and $W \in \mathbb{R}^{m \times n}$.

# Computing with multiple $x$ vectors.

Suppose we have multiple $x$ vectors, for which we want to pass through our neural network. We can stack the multiple $x$ vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

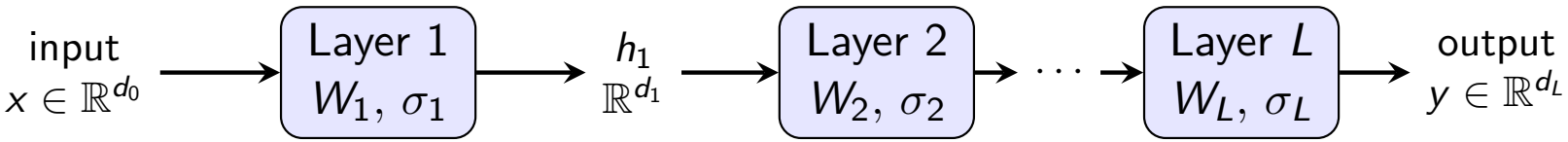
We write this succinctly as $Y = WX$, where $Y \in \mathbb{R}^{m \times B}$, $X \in \mathbb{R}^{n \times B}$, and $W \in \mathbb{R}^{m \times n}$.

## New Representation

We have successfully mapped the batched computation of a **linear layer** a matrix multiplication!

# Composing Layers

We can view $W$ as a function that takes in an $n$-dimensional vector and maps it to an $m$-dimensional vector. $W : \mathbb{R}^n \to \mathbb{R}^m$. We create deep neural networks by *composing* these transformations, along with **nonlinearities** $\sigma_i$.




$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}), \qquad y = h_L$$

# Takeaways

1. A linear layer applied to a batch of inputs can be expressed as a single matrix multiplication, $Y = WX$.
2. Deep networks are composed of many of these linear transformations.
3. **Matrix multiplication therefore captures a large and reusable part of neural network computation.**

**Why is this representation useful?**

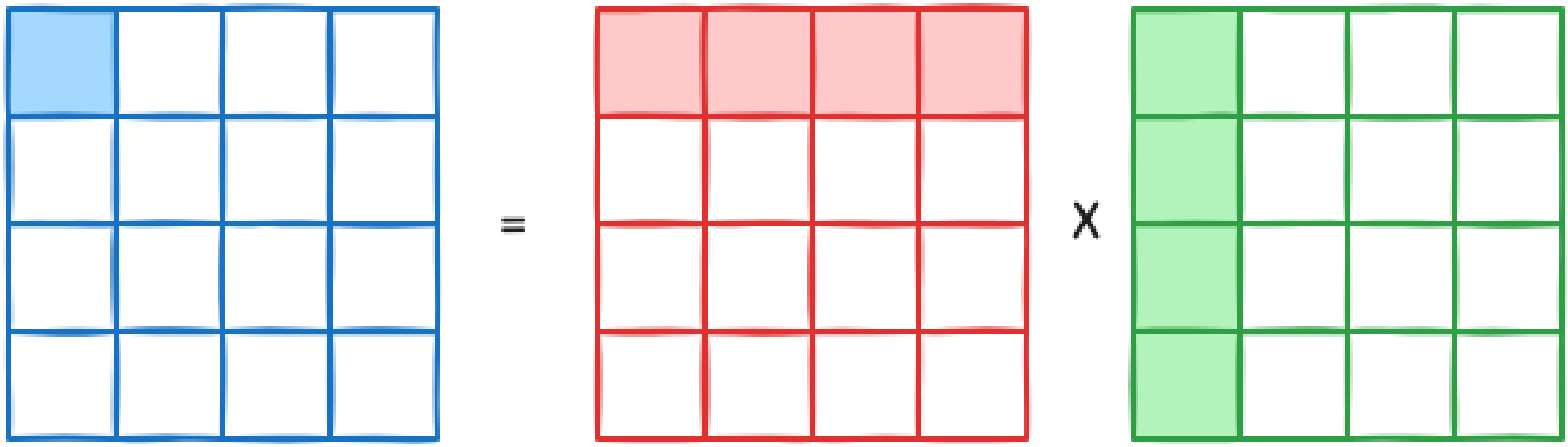
By optimizing a the MATMUL, we also optimize neural networks.

# Table of Contents


1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

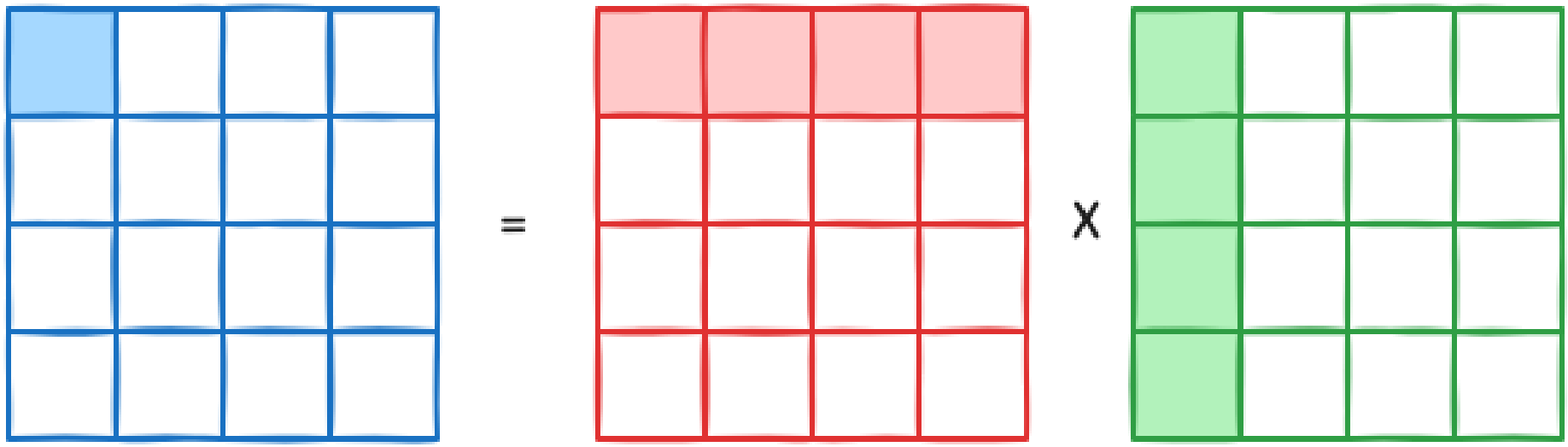
# Computing a Matrix Multiplication

Let $A$ and $B$ be two $4 \times 4$ matrices. The matrix multiple $C = AB$ results in another $4 \times 4$ matrix.



# Computing a Matrix Multiplication

Let $A$ and $B$ be two $4 \times 4$ matrices. The matrix multiple $C = AB$ results in another $4 \times 4$ matrix.



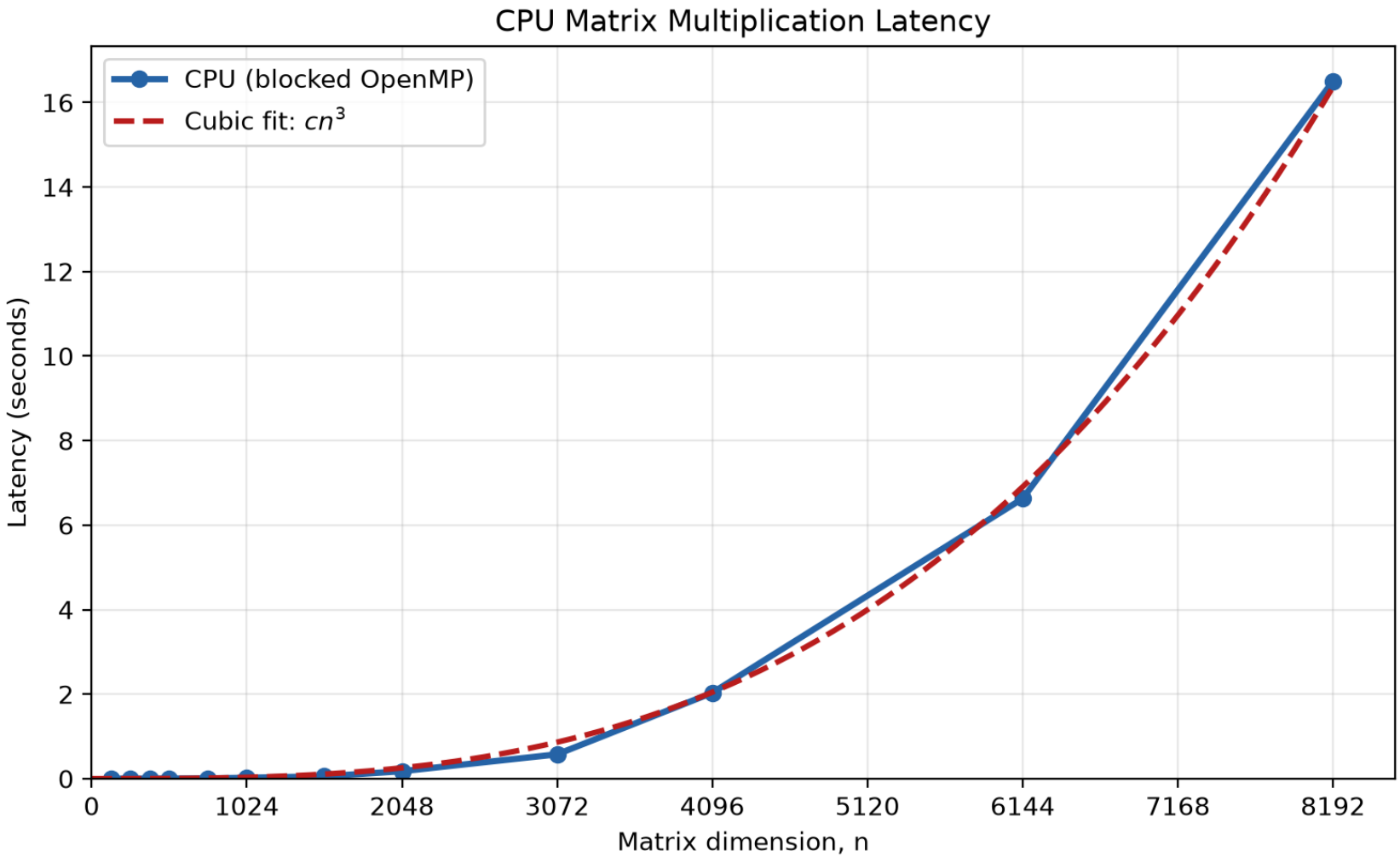

## Poor scaling

The operations are very simple, but the number of operations grow on the order of $\mathcal{O}(n^3)$, where $n$ is the dimensions of the matrices.

# Runtime of a matrix multiplication on a CPU

```
for (std::size_t y = 0; y < n; ++y) {
  for (std::size_t x = 0; x < n; ++x) {
    double sum = 0.0;
    for (std::size_t k = 0; k < n; ++k) {
      sum += a[y*n+k] * b[k*n+x];
    }
    c[y*n+x] = sum;
  }
}
```

# Runtime of a matrix multiplication on a CPU



# Why Are GPUs Massively Parallel?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

# Why Are GPUs Massively Parallel?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

## Graphics exposes data parallelism

Most vertices and pixels can be processed independently. The GPU can run the same operations on many different pieces of data at once.

# Why Are GPUs Massively Parallel?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

## Graphics exposes data parallelism

Most vertices and pixels can be processed independently. The GPU can run the same operations on many different pieces of data at once.

A CPU uses a small number of sophisticated cores to minimize the latency of one instruction stream. A GPU instead uses many simpler execution units to maximize *throughput*.

# Why Are GPUs Massively Parallel?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

## Graphics exposes data parallelism

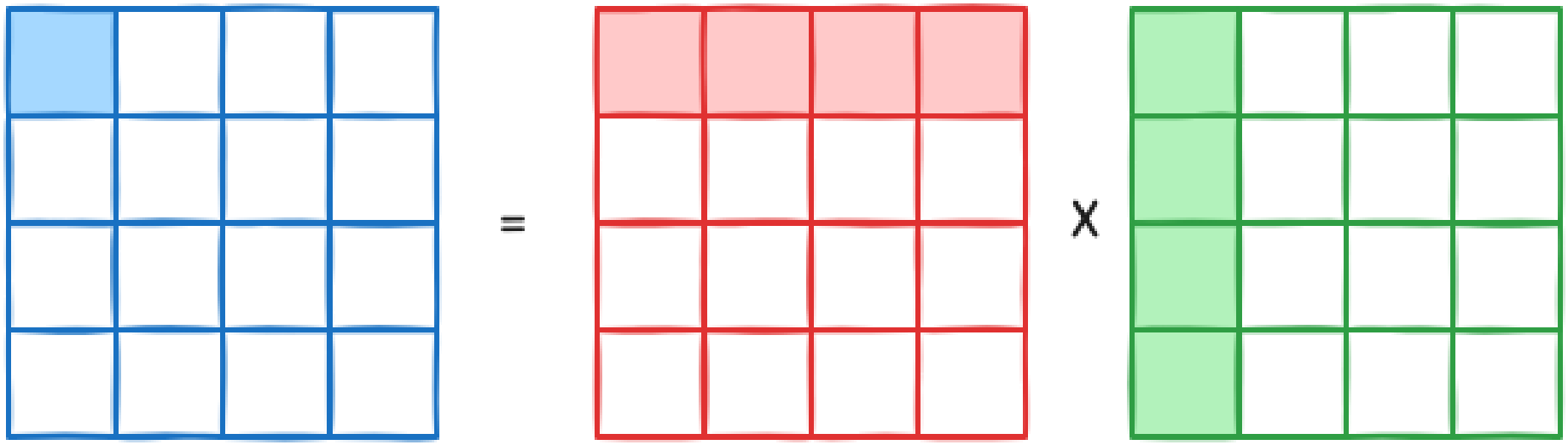
Most vertices and pixels can be processed independently. The GPU can run the same operations on many different pieces of data at once.

A CPU uses a small number of sophisticated cores to minimize the latency of one instruction stream. A GPU instead uses many simpler execution units to maximize *throughput*.

## The key tradeoff

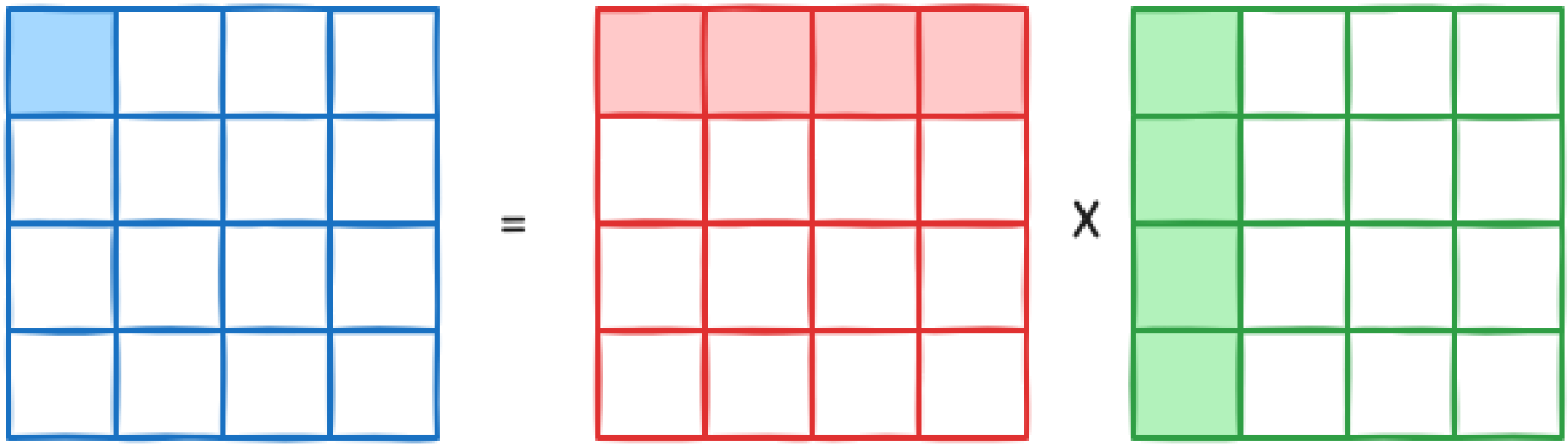
Graphics rendering is difficult because of the **volume** of similar computations, not because any one computation is especially complex.

# Matrix Multiplication is Parallel



Each element of the output is independent. The results of $C_{11}$ does not effect the results of $C_{12}$, or any other element.

# Matrix Multiplication is Parallel



Each element of the output is independent. The results of $C_{11}$ does not effect the results of $C_{12}$, or any other element.

## Key Idea

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication?

# Takeaway

- CPUs can't efficiently perform matrix multiplication at scale.
- Graphics rendering is a parallel task, and so is matrix multiplication. Thus, GPUs can be good at matrix multiplication.
- GEMM is a generalization of of matrix multiplication, and is the operation we will be discussing.

## Whats next

I've suggested that GPUs *can* do matrix multiplication, but how do we actually get there?

# Table of Contents


1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

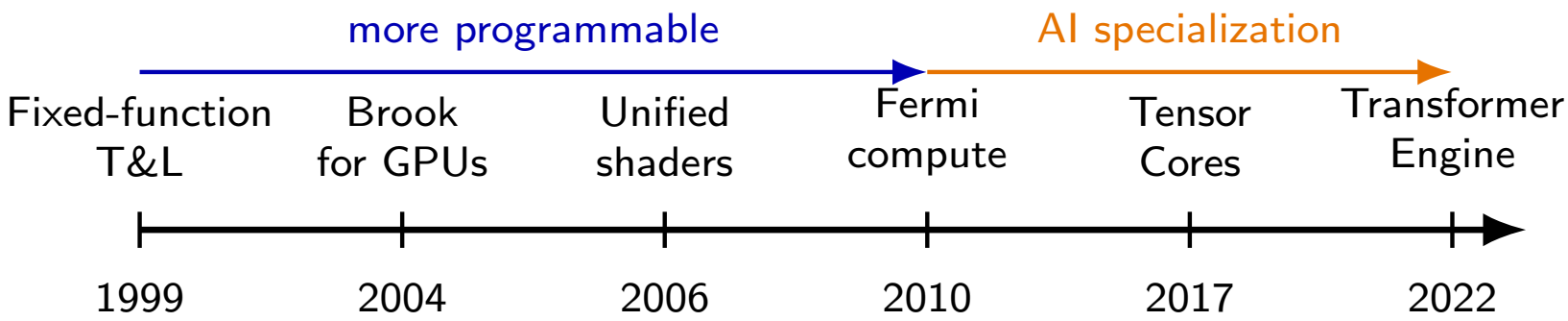
# Timeline

- **1999—2003:** fixed-function graphics gives way to programmable vertex and pixel shaders.
- **2004:** Brook for GPUs demonstrates a high-level stream-programming model on top of graphics APIs.
- **2006—2007:** GeForce 8800 introduces a unified shader architecture; NVIDIA releases CUDA publicly in 2007.
- **2010:** Fermi adds a coherent cache hierarchy, ECC support, and much stronger support for C++ and scientific computing.
- **2017:** Volta introduces Tensor Cores for mixed-precision matrix multiply–accumulate.
- **2020—2024:** Ampere broadens Tensor Core formats; Hopper adds the FP8 Transformer Engine; Blackwell adds further low-precision and AI-specific hardware.

# What shaped modern GPU architecture?

## Driver of architectural change

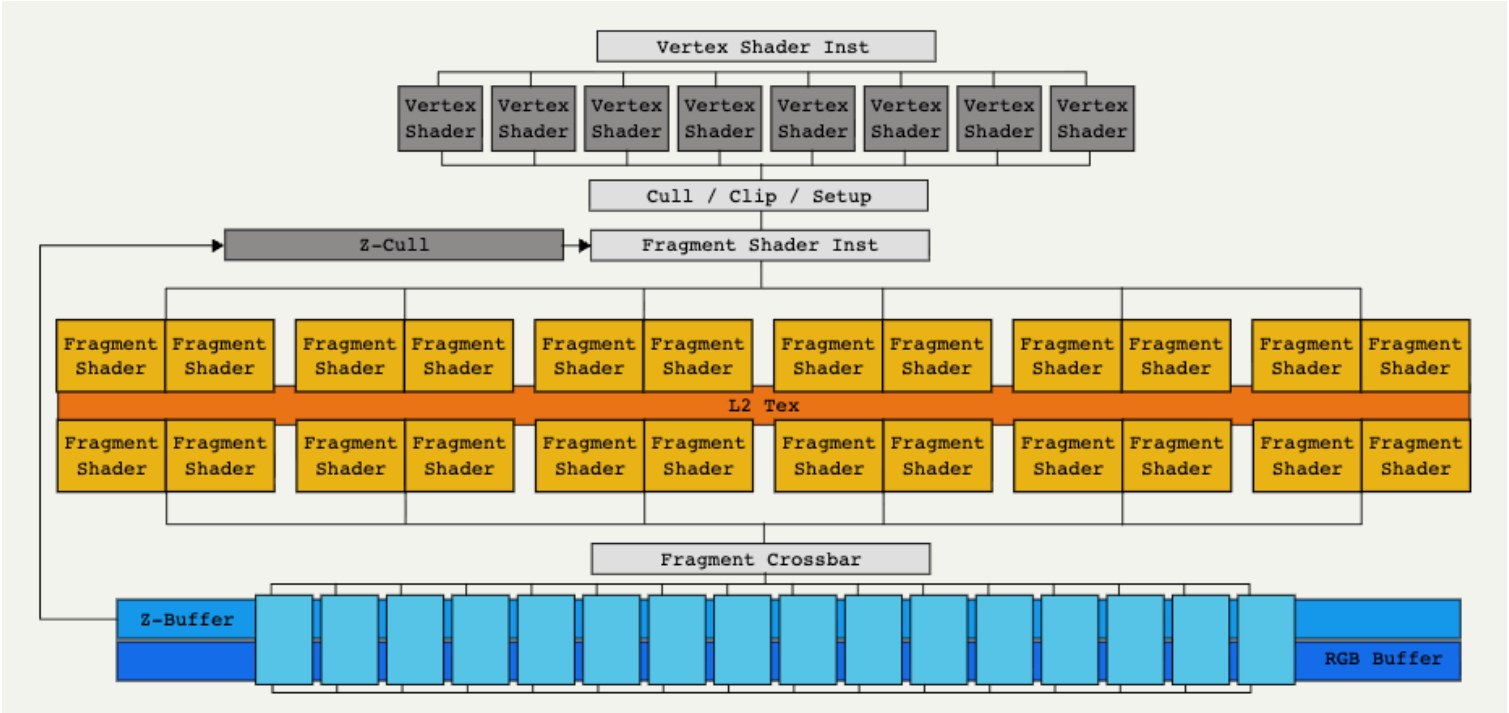
New workloads cause generalization/programmability, established workloads cause specialization.



# What shaped modern GPU architecture?

## <2006: graphics first

- GeForce 256 (1999) accelerated fixed-function transform and lighting.
- Later chips added programmable vertex and pixel shaders, but kept distinct stages and resources.




The G71 architecture (2005). Diagram source: Sanglard (2020).

# What shaped modern GPU architecture?

**2004: GPGPU before CUDA. Demand was always there.**

- Brook for GPUs compiled a stream language to graphics APIs.
- It showed useful non-graphics workloads could exploit GPU throughput, despite an awkward programming model.

**Brook for GPUs: Stream Computing on Graphics Hardware**

Ian Buck Tim Foley Daniel Horn Jeremy Sugerman Kayvon Fatahalian Mike Houston Pat Hanrahan
Stanford University

**Abstract**

In this paper, we present Brook for GPUs, a system for general-purpose computation on programmable graphics hardware. Brook extends C to include simple data-parallel constructs, enabling the use of the GPU as a streaming co-processor. We present a compiler and runtime system that abstracts and virtualizes many aspects of graphics hardware. In addition, we present an analysis of the effectiveness of the GPU as a compute engine compared to the CPU, to determine when the GPU can outperform the CPU for a particular algorithm. We evaluate our system with five applications, the SAXPY and SGEMV BLAS operators, image segmentation, FFT, and ray tracing. For these applications, we demonstrate that our Brook implementations perform comparably to hand-written GPU code and up to seven times faster than their CPU counterparts.

**CR Categories:** I.3.1 [Computer Graphics]: Hardware Architecture—Graphics processors D.3.2 [Programming Languages]: Language Classifications—Parallel Languages

**Keywords:** Programmable Graphics Hardware, Data Parallel Computing, Stream Computing, GPU Computing, Brook

modern hardware. In addition, the user is forced to express their algorithm in terms of graphics primitives, such as textures and triangles. As a result, general-purpose GPU computing is limited to only the most advanced graphics developers.

This paper presents *Brook*, a programming environment that provides developers with a view of the GPU as a streaming coprocessor. The main contributions of this paper are:

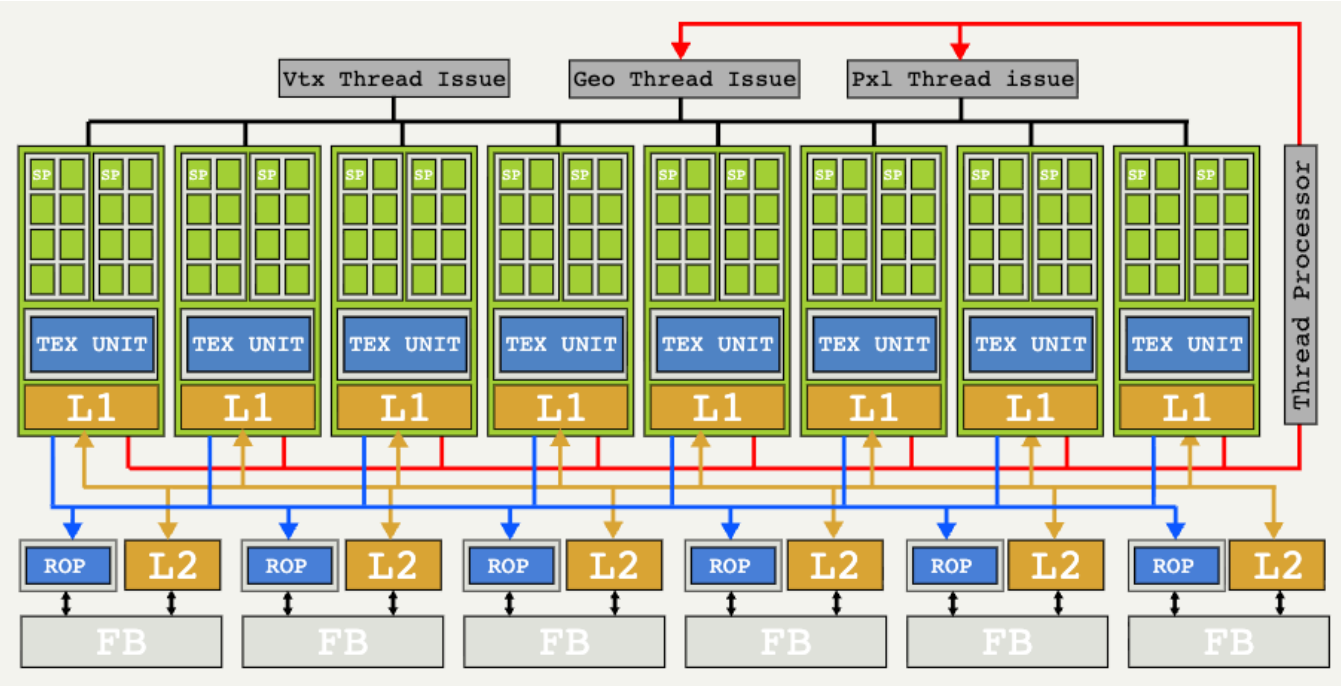
- The presentation of the Brook stream programming model for general-purpose GPU computing. Through the use of streams, kernels and reduction operators, Brook abstracts the GPU as a streaming processor.
- The demonstration of how various GPU hardware limitations can be virtualized or extended using our compiler and runtime system; specifically, the GPU memory system, the number of supported shader outputs, and support for user-defined data structures.
- The presentation of a cost model for comparing GPU vs. CPU performance tradeoffs to better understand under what circumstances the GPU outperforms the CPU.

Buck et al., "Brook for GPUs," *ACM TOG* 23(3), 2004.

# What shaped modern GPU architecture?

**2006-2007: increasing complexity necessitates unified compute**

- G80 ran vertex, geometry, and pixel work on unified pool of **Streaming Multiprocessors** (SM's).
- **This is the heart of modern GPU architecture.**




G80 block diagram. Diagram source: Sanglard (2020); architecture described by Lindholm et al. (2008).

# Our model of the GPU (Execution)

The GPU has a large pool of many **SM**s that execute independently from each other. Each SM has many **CUDA cores** that perform arithmetic operations.

# Our model of the GPU (Execution)

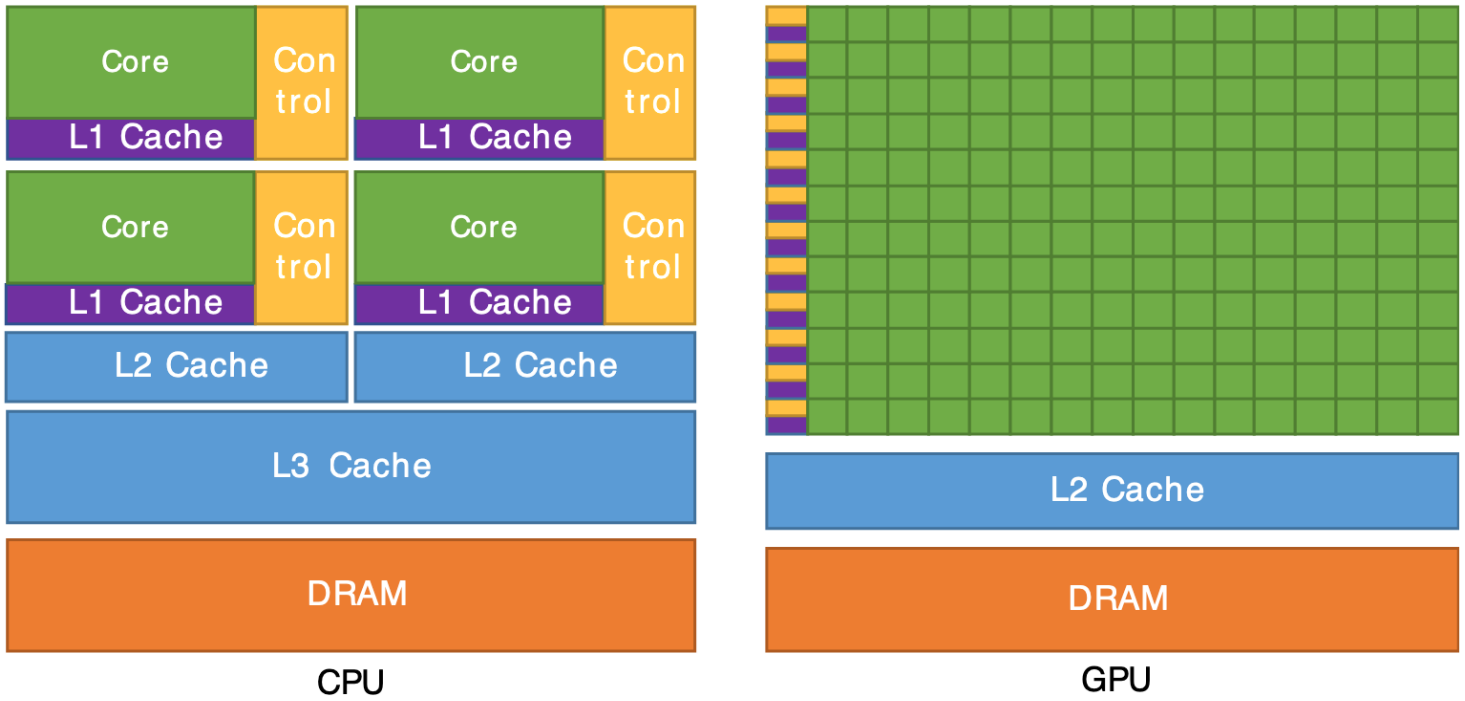
The GPU has a large pool of many **SM**s that execute independently from each other. Each SM has many **CUDA cores** that perform arithmetic operations.

**The CUDA Cores execute the same instruction in parallel on different data**.

# Our model of the GPU (Execution)

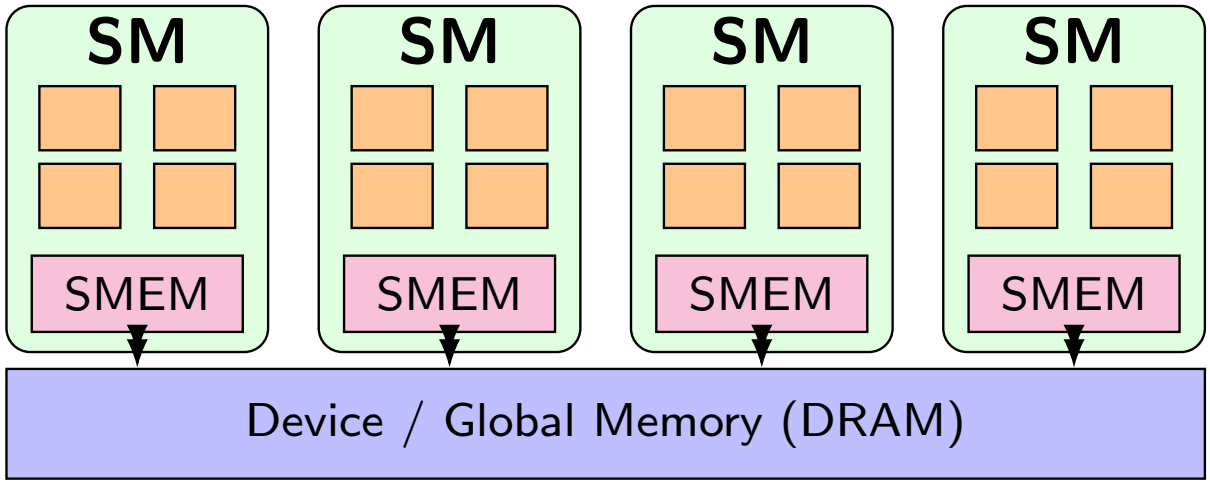
The GPU has a large pool of many **SM**s that execute independently from each other. Each SM has many **CUDA cores** that perform arithmetic operations.

**The CUDA Cores execute the same instruction in parallel on different data**.



# Our model of the GPU (Memory)

The GPU has its own memory called **Device Memory (DRAM)**. Each SM has a **programmable L1 Cache** which we also call **Shared Memory (SMEM)**.

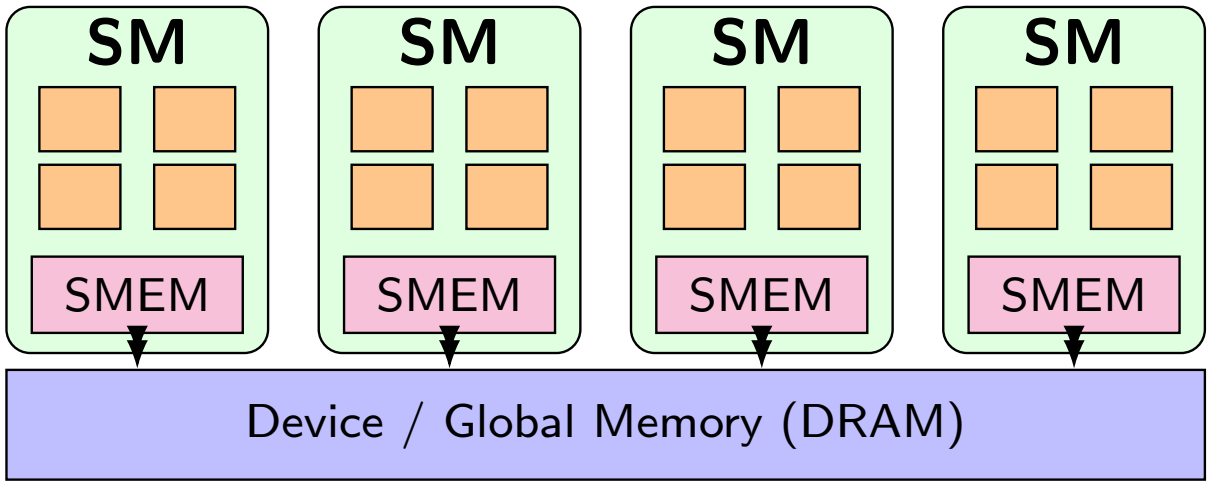



## Performance

Accessing to DRAM is orders of **tens of times** slower than accessing SMEM. Effective use of SMEM is crucial to performance.

# Takeaways

- SMs: primary compute unit
- CUDA Cores: like the "ALUs" of the SMs.
- L1 cache/Shared memory: cache for the SM
- Device/Global Memory: like RAM for a normal CPU system.




## Whats next

How can we actually implement a matrix multiplication on a GPU? What tools and mental models do we need?

# Table of Contents


1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

# Section Overview


1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

# What is GEMM?

**Definition: Single-precision general matrix multiplication (SGEMM)**

$$C \leftarrow \alpha AB + \beta C$$

# What is GEMM?

**Definition: Single-precision general matrix multiplication (SGEMM)**

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

# What is GEMM?

## Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

## Example

During the forward pass when training a model, we accumulate the gradients from the entire batch.

# What is GEMM?

## Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

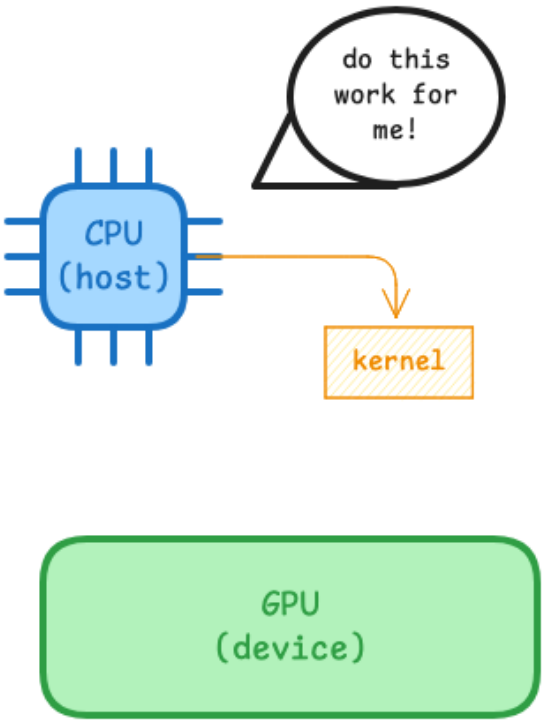
```
void launch_sgemm(const SgemmKernel
    kernel, const int M, const int N,
                  const int K, const
    float alpha, const float *A,
                  const float *B, const
    float beta, float *C,
                  const cudaStream_t
    stream = nullptr);
```

$$\underbrace{A}_{M\times K} \quad \underbrace{B}_{K\times N} \quad \underbrace{C}_{M\times N}$$

- $M$: rows of $A$ and $C$
- $N$: columns of $B$ and $C$
- $K$: shared reduction dimension

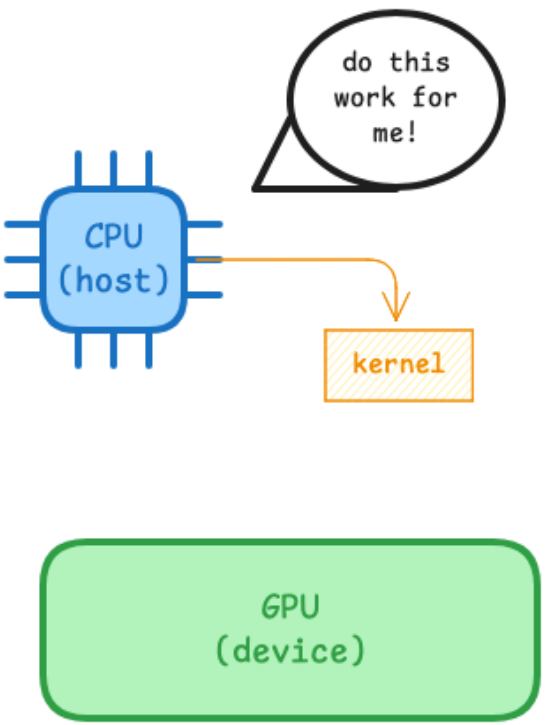
# CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.



# CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.




```
int main() {
  const int M = 128, N = 128, K = 128;

  float *dA, *dB, *dC;
  cudaMalloc(&dA, M * K * sizeof(float);
  cudaMalloc(&dB, K * N * sizeof(float));
  cudaMalloc(&dC, M * N * sizeof(float));

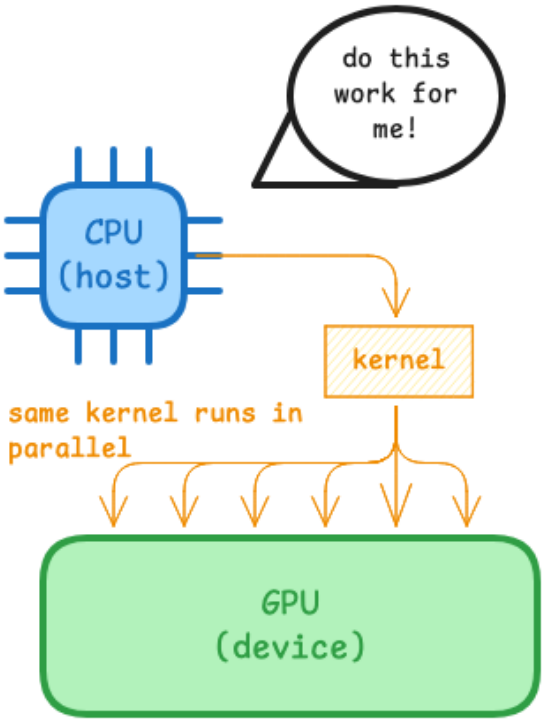
  // load values into these device pointers...

  run_sgemm_naive(M, N, K, 1, dA, dB, 0, dC);

  cudaFree(dA); cudaFree(dB); cudaFree(dC);
}
```

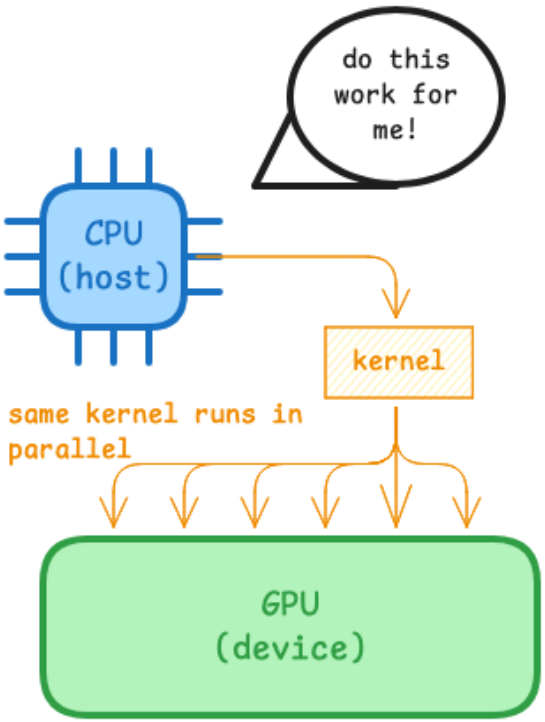
# CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.



# CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.



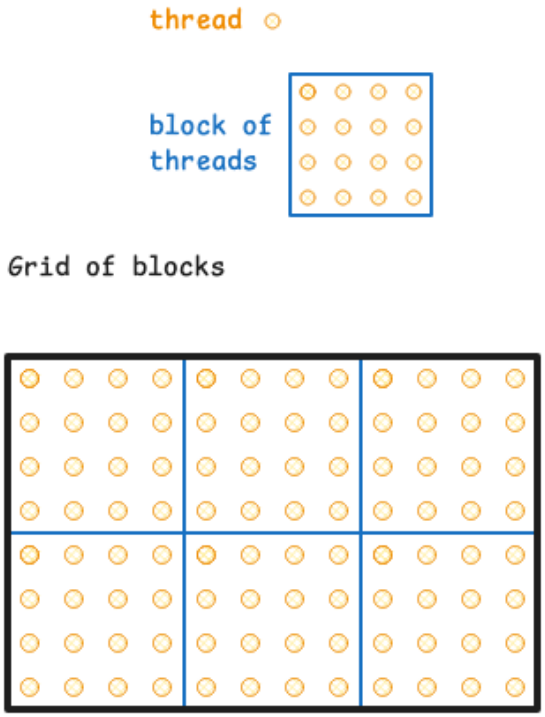

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float
      *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}

// Host-side launch configuration:

    constexpr int block_size = 32;
    const dim3 blockDim(block_size, block_size);
    const dim3 gridDim((N + block_size - 1) / block_size,
                       (M + block_size - 1) / block_size);
    sgemm_naive<<<gridDim, blockDim, 0, stream>>>(M, N, K, alpha, A, B,
      beta,
                                                  C);
```

# CUDA Basics (Threads, Blocks, Grids)

- Threads are grouped into **blocks**.
- The collection of blocks requested by the host is organized in a **grid**.




```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float
      *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}

// Host-side launch configuration:

    constexpr int block_size = 32;
    const dim3 blockDim(block_size, block_size);
    const dim3 gridDim((N + block_size - 1) / block_size,
                       (M + block_size - 1) / block_size);
    sgemm_naive<<<gridDim, blockDim, 0, stream>>>(M, N, K, alpha, A, B,
      beta,
                                                  C);
```

# Why Threads, Blocks, Grids?

**This seems complicated, why can't we just ask for $N$ threads?**

# Why Threads, Blocks, Grids?

**This seems complicated, why can't we just ask for $N$ threads?**

## Relation to hardware

This hierarchy is closely related to how the device schedules the execution. We will see later how to use this to optimize performance.

# Why Threads, Blocks, Grids?

**This seems complicated, why can't we just ask for $N$ threads?**

## Relation to hardware

This hierarchy is closely related to how the device schedules the execution.
We will see later how to use this to optimize performance.

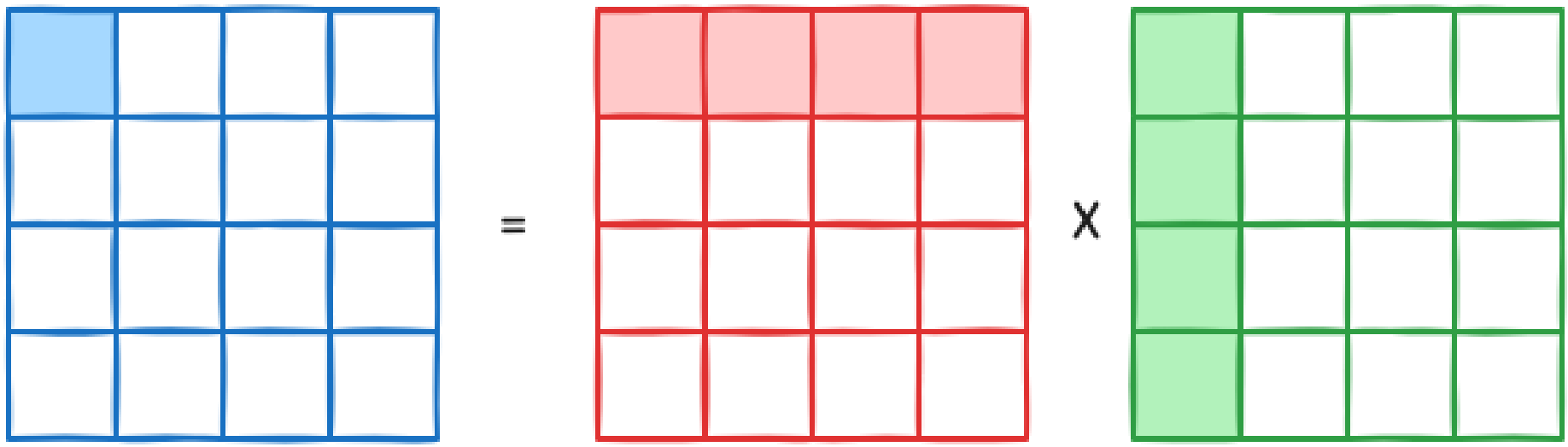
## Thread, Block, Grid hierarchy

Threads are grouped into blocks of constant size/shape.
Blocks are launched as a grid.
We choose the dimensions of the grid to control how many blocks are launched, and thus how many threads are launched.

# Let's design a SGEMM kernel!

## Recall

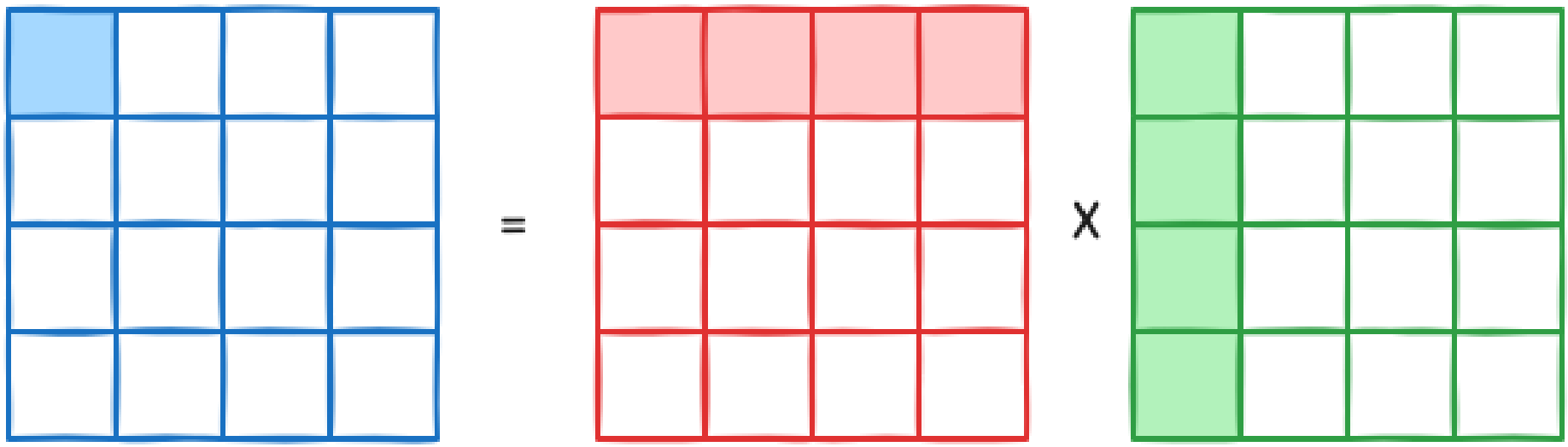
Recall that matrix multiplication is naturally parallel, and the output elements of $C$ can be computed independently in parallel.



# Let's design a SGEMM kernel!

## Recall

Recall that matrix multiplication is naturally parallel, and the output elements of $C$ can be computed independently in parallel.



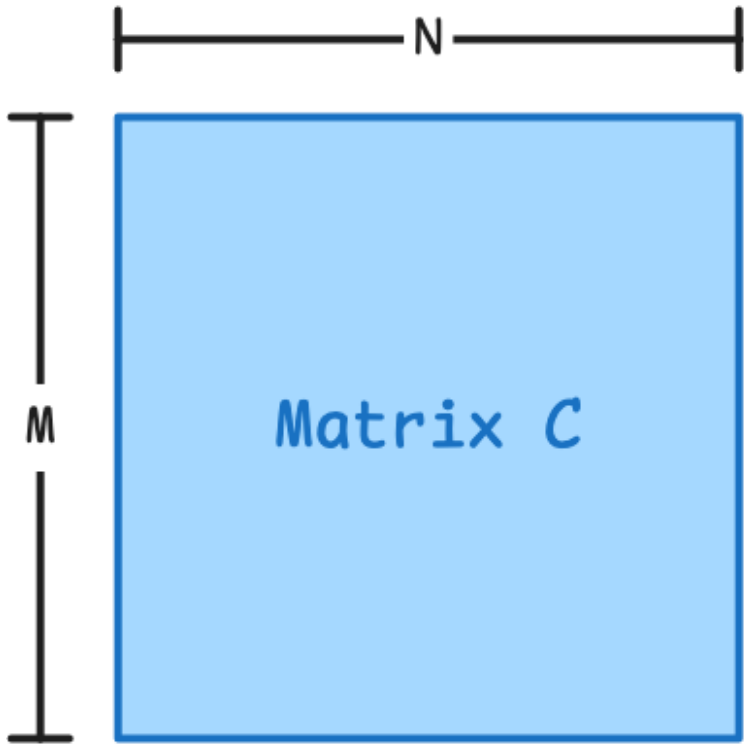
**We can naturally map one thread to each output element!**

# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int
      K,
                            const float alpha, const float *A,
      const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0.0f;
    for (int i = 0; i < K; ++i) {
      sum += A[y * K + i] * B[i * N + x];
    }
    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
// Host-side launch configuration:

    constexpr int block_size = 32;
    const dim3 blockDim(block_size, block_size);
    const dim3 gridDim((N + block_size - 1) / block_size,
                       (M + block_size - 1) / block_size);
    sgemm_naive<<<gridDim, blockDim, 0, stream>>>(M, N, K, alpha
      , A, B, beta,
                                                  C);
```

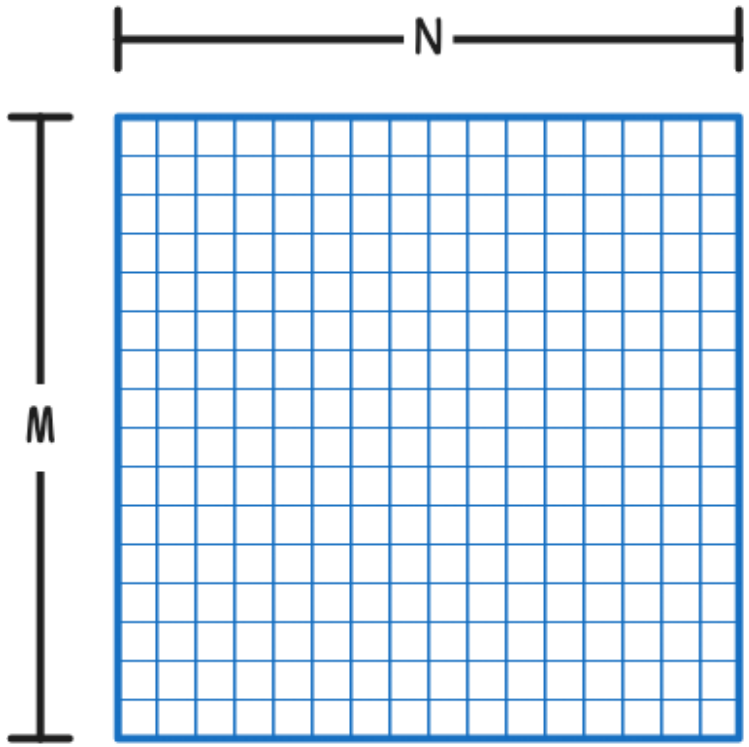


# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int
      K,
                            const float alpha, const float *A,
      const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0.0f;
    for (int i = 0; i < K; ++i) {
      sum += A[y * K + i] * B[i * N + x];
    }
    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
// Host-side launch configuration:

    constexpr int block_size = 32;
    const dim3 blockDim(block_size, block_size);
    const dim3 gridDim((N + block_size - 1) / block_size,
                       (M + block_size - 1) / block_size);
    sgemm_naive<<<gridDim, blockDim, 0, stream>>>(M, N, K, alpha
      , A, B, beta,
                                                  C);
```

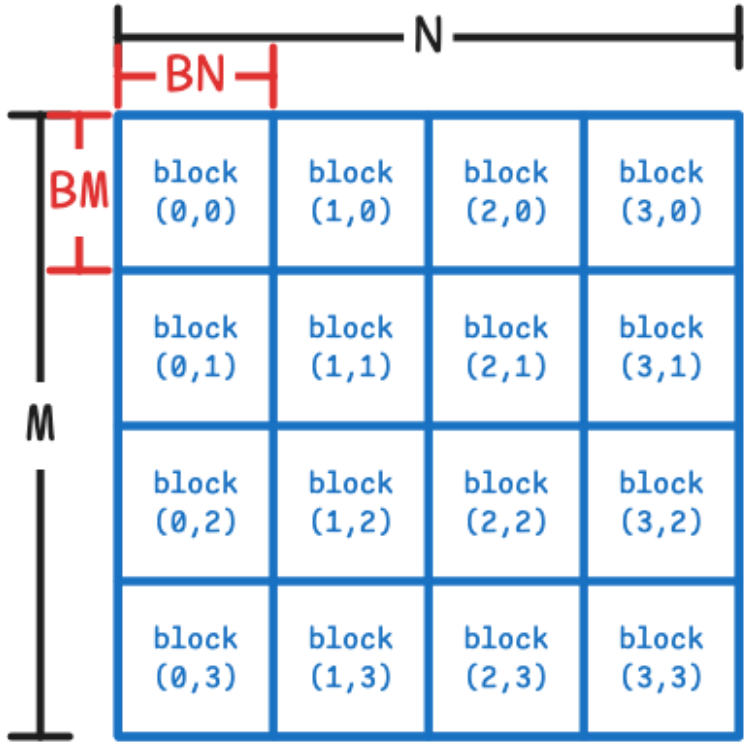


# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int
      K,
                            const float alpha, const float *A,
      const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0.0f;
    for (int i = 0; i < K; ++i) {
      sum += A[y * K + i] * B[i * N + x];
    }
    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
// Host-side launch configuration:

    constexpr int block_size = 32;
    const dim3 blockDim(block_size, block_size);
    const dim3 gridDim((N + block_size - 1) / block_size,
                       (M + block_size - 1) / block_size);
    sgemm_naive<<<gridDim, blockDim, 0, stream>>>(M, N, K, alpha
      , A, B, beta,
                                                  C);
```

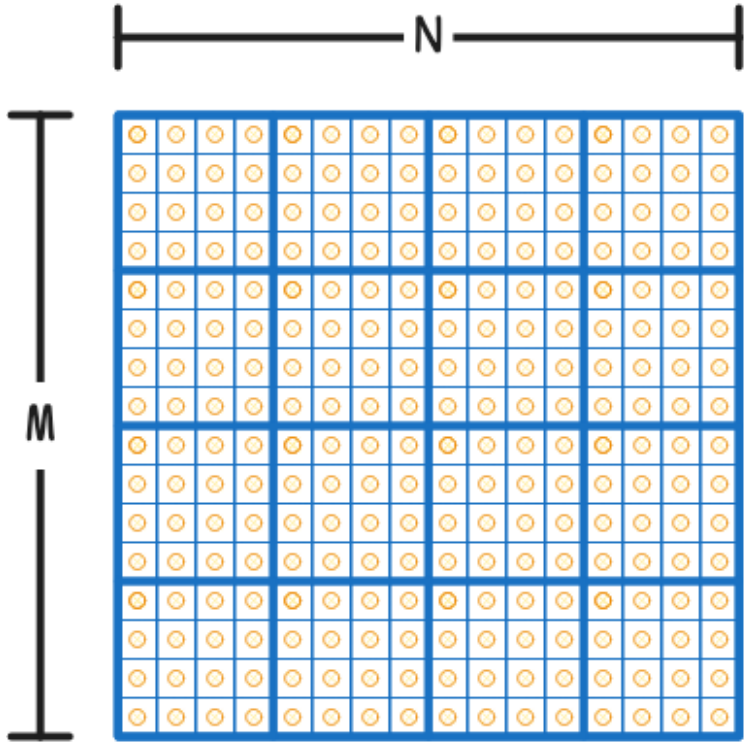


# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int
      K,
                            const float alpha, const float *A,
      const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0.0f;
    for (int i = 0; i < K; ++i) {
      sum += A[y * K + i] * B[i * N + x];
    }
    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
// Host-side launch configuration:

    constexpr int block_size = 32;
    const dim3 blockDim(block_size, block_size);
    const dim3 gridDim((N + block_size - 1) / block_size,
                       (M + block_size - 1) / block_size);
    sgemm_naive<<<gridDim, blockDim, 0, stream>>>(M, N, K, alpha
      , A, B, beta,
                                                  C);
```

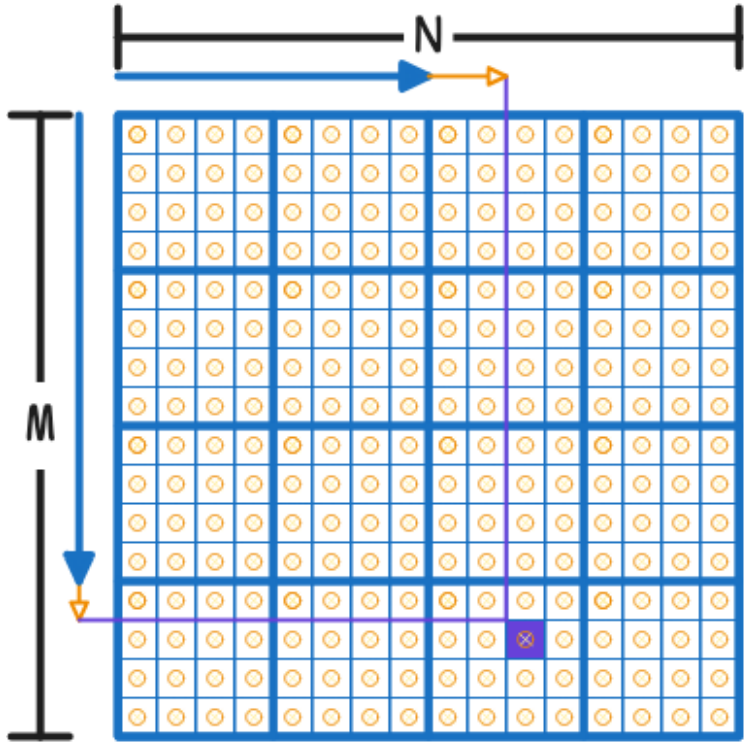


# Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int
      K,
                            const float alpha, const float *A,
      const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0.0f;
    for (int i = 0; i < K; ++i) {
      sum += A[y * K + i] * B[i * N + x];
    }
    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
// Host-side launch configuration:

    constexpr int block_size = 32;
    const dim3 blockDim(block_size, block_size);
    const dim3 gridDim((N + block_size - 1) / block_size,
                       (M + block_size - 1) / block_size);
    sgemm_naive<<<gridDim, blockDim, 0, stream>>>(M, N, K, alpha
      , A, B, beta,
                                                  C);
```

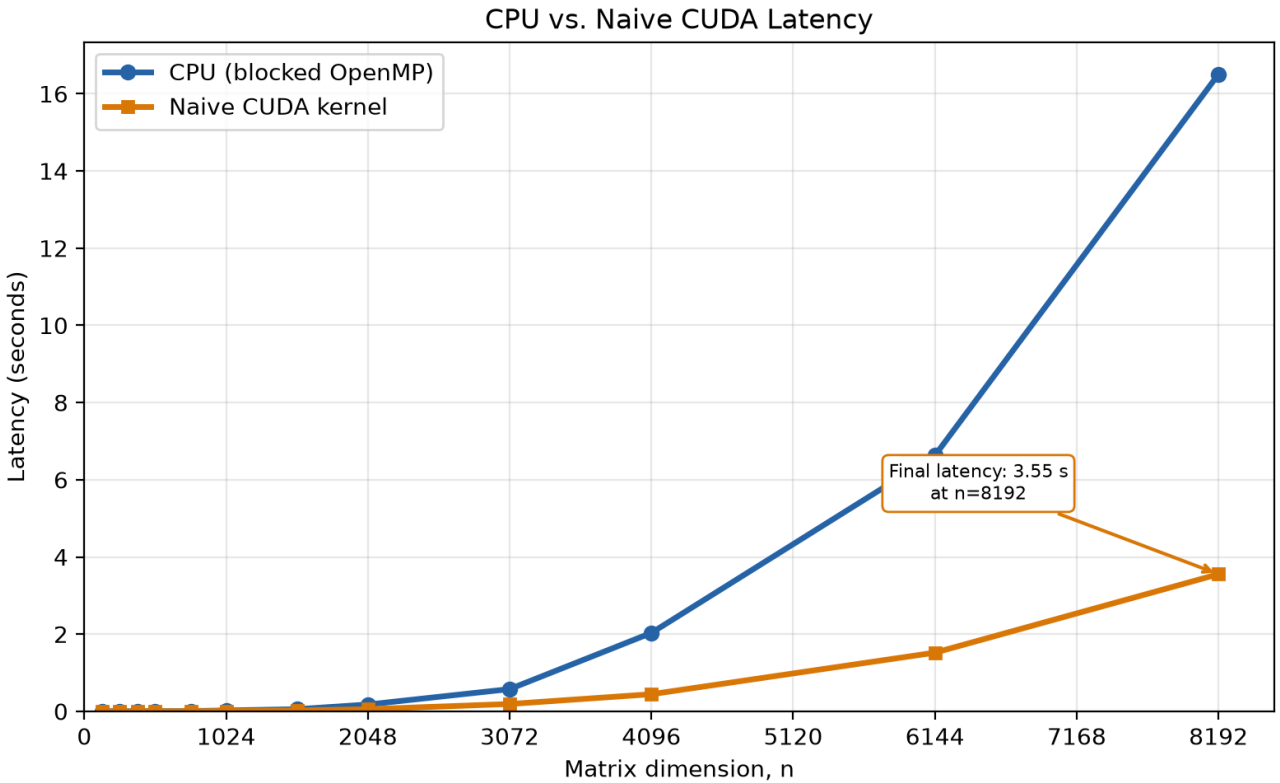


# Naive SGEMM visualization

- One thread computes one element of $C$.
- It walks one row of $A$ and one column of $B$ for the full dot product.

Play naive-kernel visualization

# Naive CUDA is already faster than the CPU



# Section Overview


1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

# We can do better

**Observed latency with naive kernel: 3.55s.**

# We can do better

**Observed latency with naive kernel: 3.55s**.
$m, n, k = 8192$ requires $mn(2k) = 2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.

# We can do better

**Observed latency with naive kernel: 3.55s**.
$m, n, k = 8192$ requires $mn(2k) = 2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.
These benchmarks were run on an RTX 3090 with **35.6 TFLOP/s** of peak FP32 **throughput**.

# We can do better

**Observed latency with naive kernel: 3.55s**.
$m, n, k = 8192$ requires $mn(2k) = 2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.
These benchmarks were run on an RTX 3090 with **35.6 TFLOP/s** of peak FP32 **throughput**.
Therefore, if we achieve peak FP32 throughput, we should have a latency

$$\frac{1.10 \text{ TFLOP}}{35.6 \text{ TFLOP/s}} \approx 0.031\text{s} = 31\text{ms}.$$

# We can do better

**Observed latency with naive kernel: 3.55s**.
$m, n, k = 8192$ requires $mn(2k) = 2(8192)^3 = 1.10 \times 10^{12}$ FLOPs.
These benchmarks were run on an RTX 3090 with **35.6 TFLOP/s** of peak FP32 **throughput**.
Therefore, if we achieve peak FP32 throughput, we should have a latency

$$\frac{1.10 \text{ TFLOP}}{35.6 \text{ TFLOP/s}} \approx 0.031\text{s} = 31\text{ms}.$$

## Whats the hold up?

Our observed latency of 3.55 *seconds* is more than 100$\times$ the theoretical minimum of 31 *milliseconds*!

# We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.

# We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.
We assigned a thread to each output element, and each thread reads $k$ elements from $A$ and $B$, so we have $mn(2k) = 2(8192)^3$ DRAM accesses.

# We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.
We assigned a thread to each output element, and each thread reads $k$ elements from $A$ and $B$, so we have $mn(2k) = 2(8192)^3$ DRAM accesses.
At FP32, that is $2(8192)^3 \times 4$ bytes $\approx 3298.5$ GB.

# We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.

We assigned a thread to each output element, and each thread reads $k$ elements from $A$ and $B$, so we have $mn(2k) = 2(8192)^3$ DRAM accesses.

At FP32, that is $2(8192)^3 \times 4$ bytes $\approx 3298.5$ GB.

The RTX 3090 has a DRAM bandwidth of 935.8 GB/s. So the time required to serve all the DRAM requests is $3298.5/935.8 \approx \boxed{3.52\text{s}}$.

# We aren't considering memory bandwidth.

Every thread is **reading everything from DRAM** for each element it needs.
We assigned a thread to each output element, and each thread reads $k$ elements from $A$ and $B$, so we have $mn(2k) = 2(8192)^3$ DRAM accesses.
At FP32, that is $2(8192)^3 \times 4$ bytes $\approx 3298.5$ GB.
The RTX 3090 has a DRAM bandwidth of 935.8 GB/s. So the time required to serve all the DRAM requests is $3298.5/935.8 \approx \boxed{3.52\text{s}}$.

**We are memory bound!**

Our observed latency of 3.55 seconds is dominated by the time it takes to move the data from DRAM to the CUDA Cores, so we are **memory bound**. Our CUDA Cores are sitting idle for most of the time!

# Shared Memory (SMEM) Caching

## Programmable L1 Cache (SMEM)

Remember how our launch config required us to group threads as blocks? That is because **blocks** are scheduled to SM's as a unit, so **threads in the same block have access to the SM's L1 cache/SMEM.**

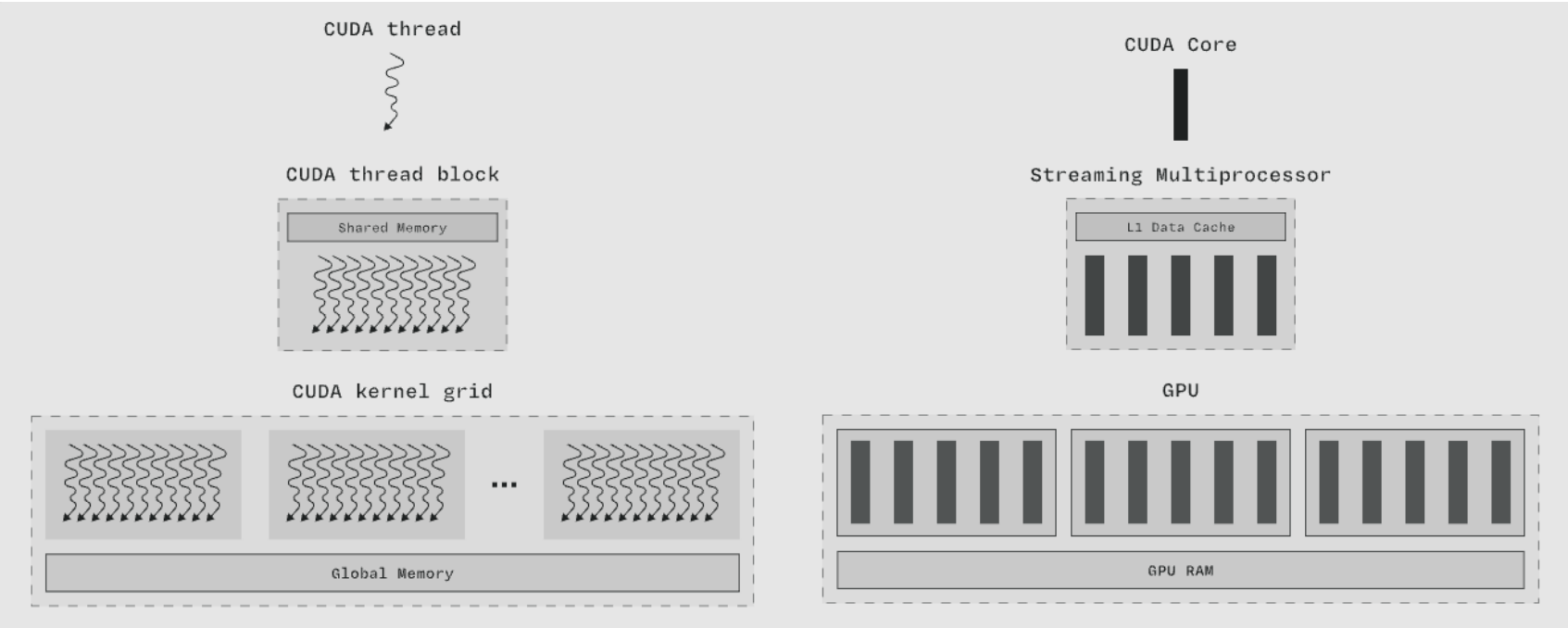



Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

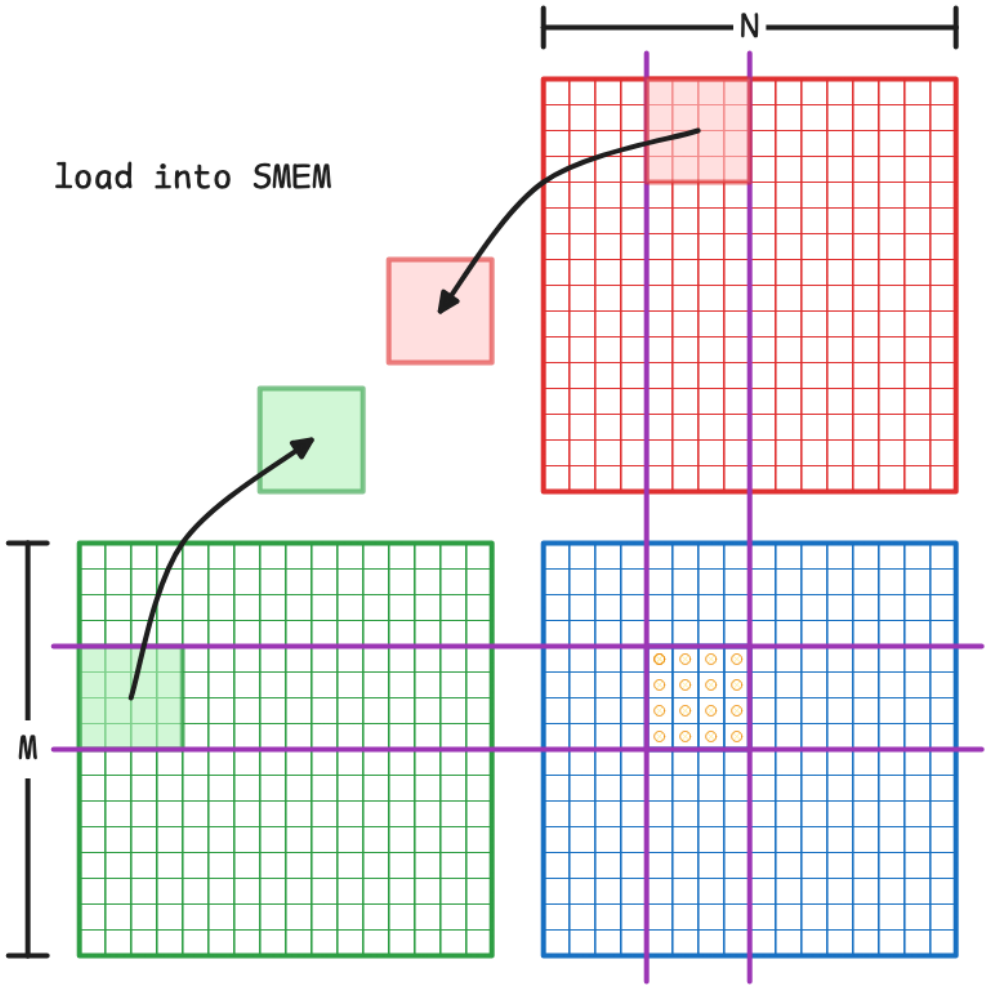
# Tiling

Threads can save time by loading data from Device/Global memory into Shared Memory within the SM.

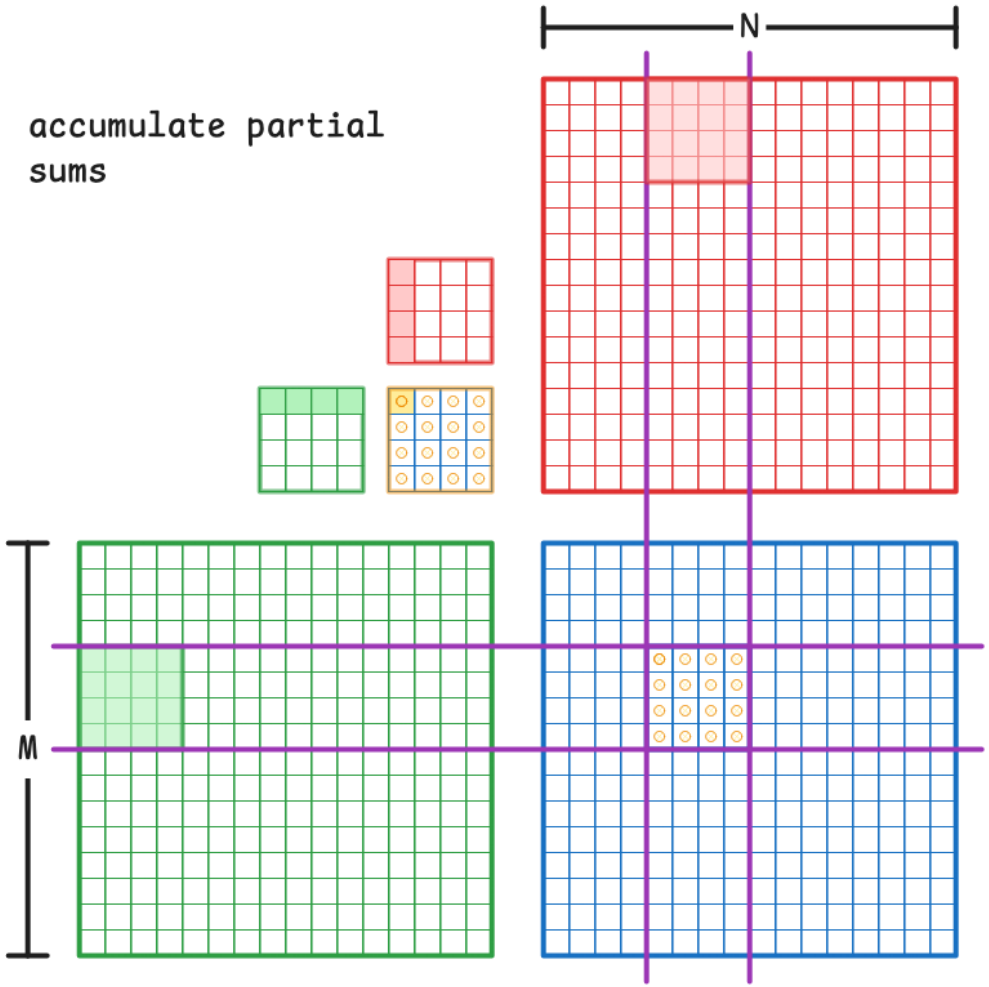
## Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.

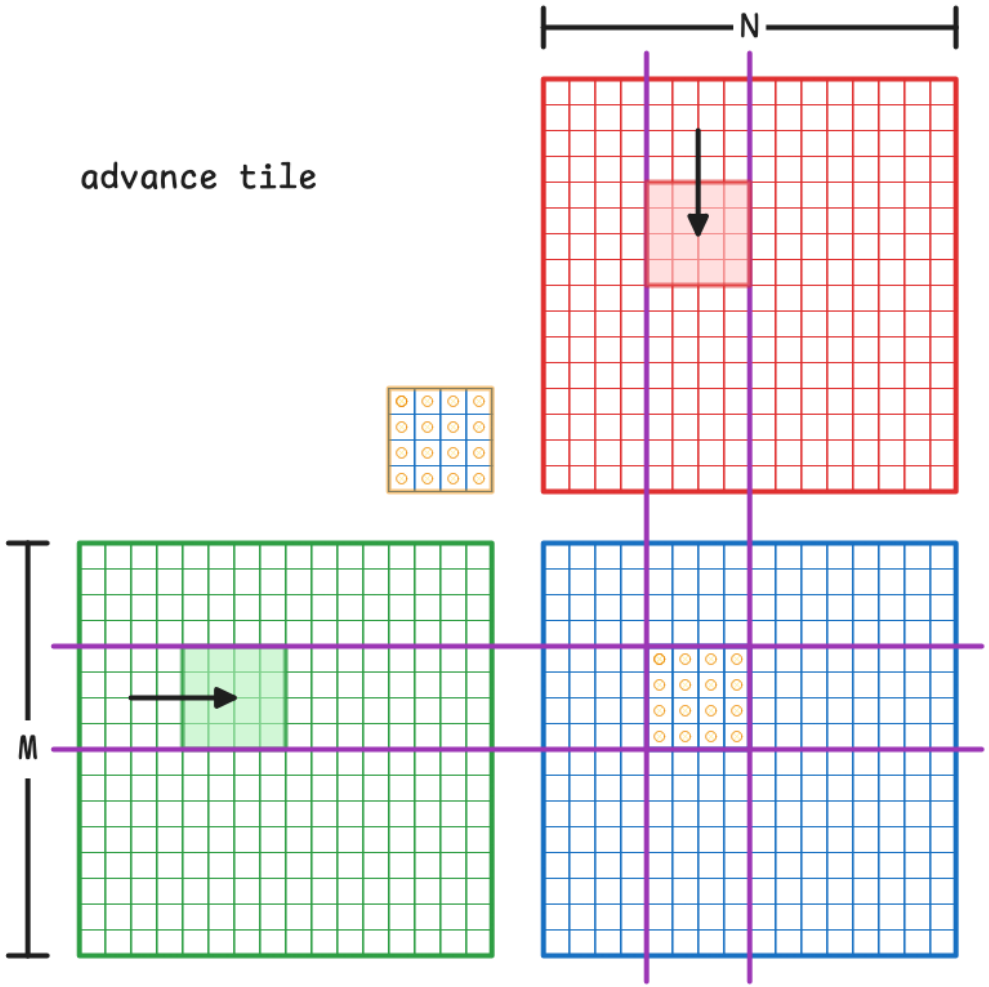
# Tiling steps



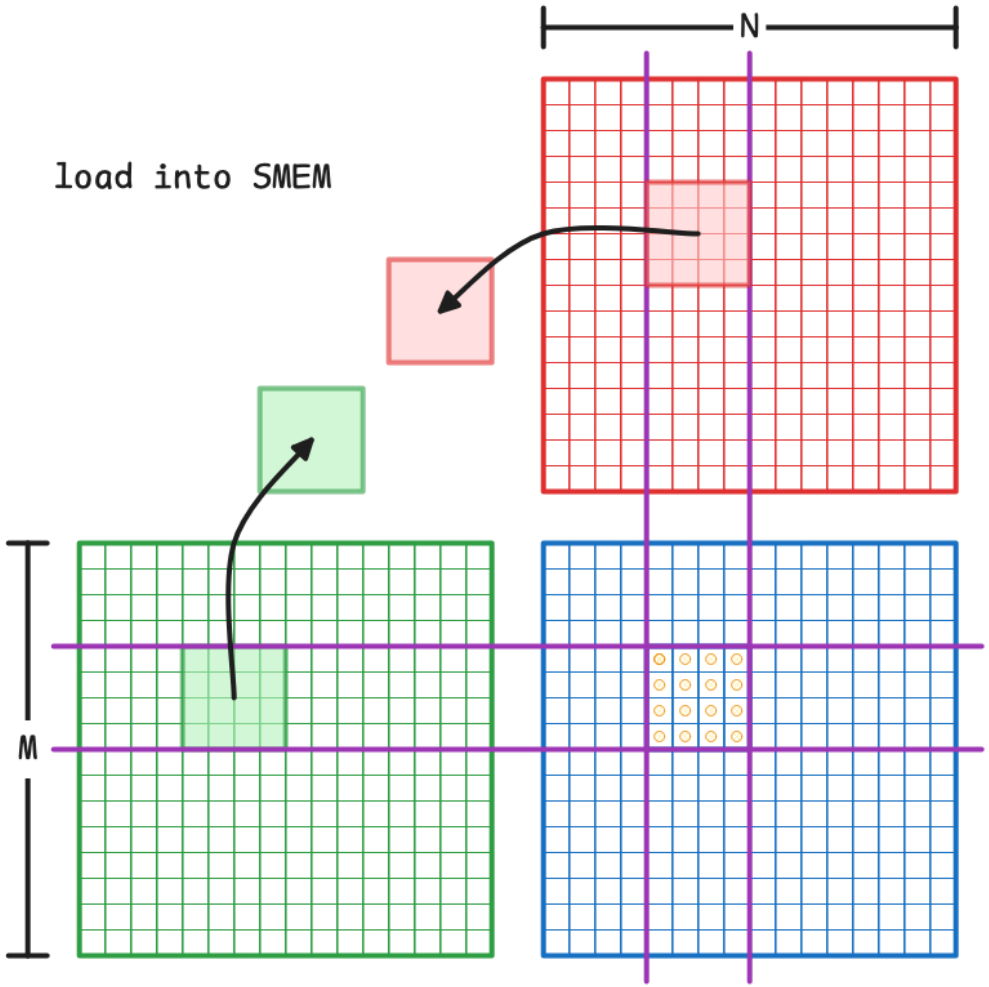
# Tiling steps



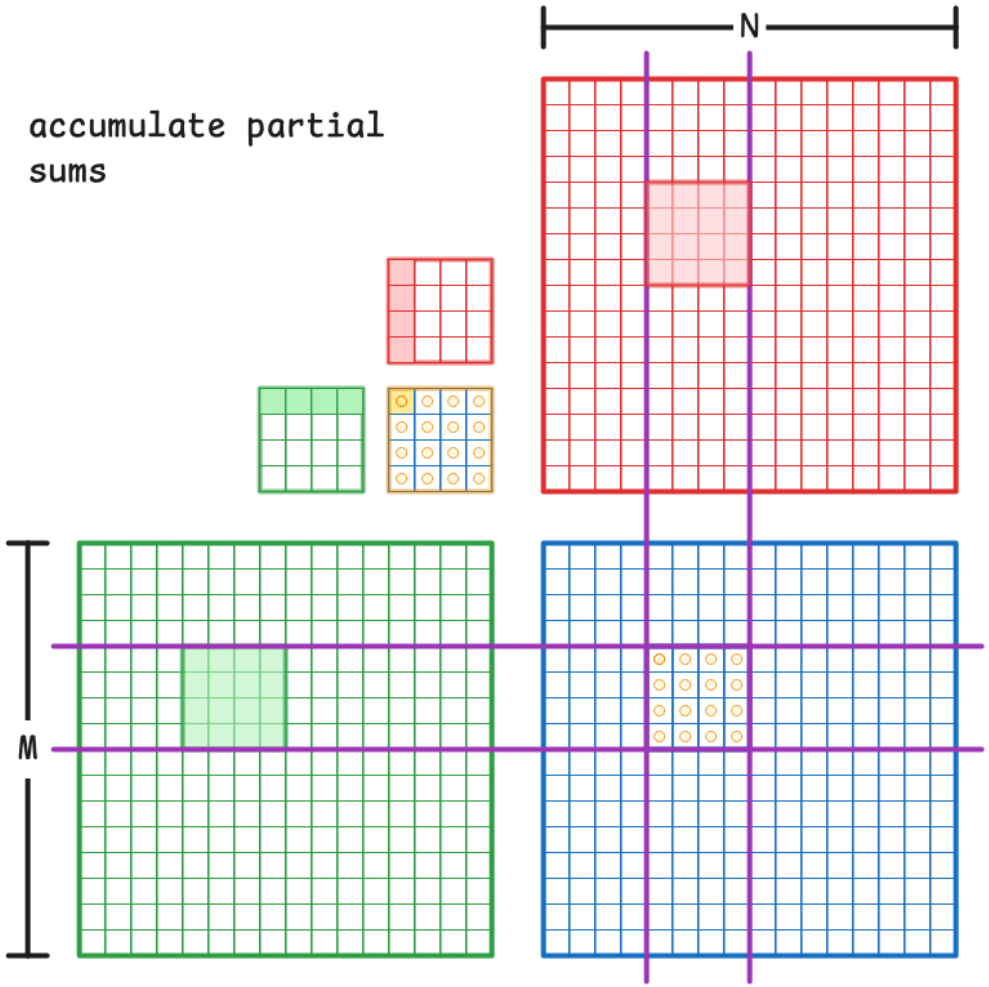
# Tiling steps



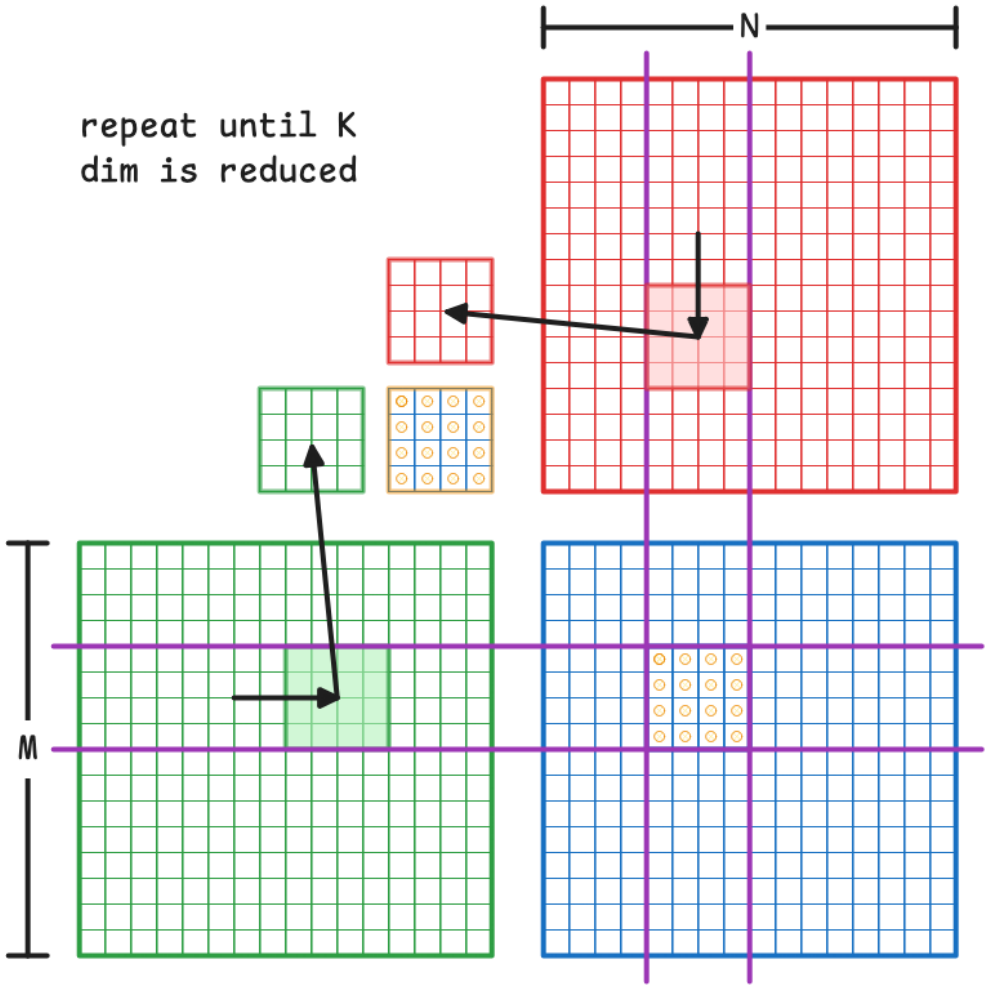
# Tiling steps



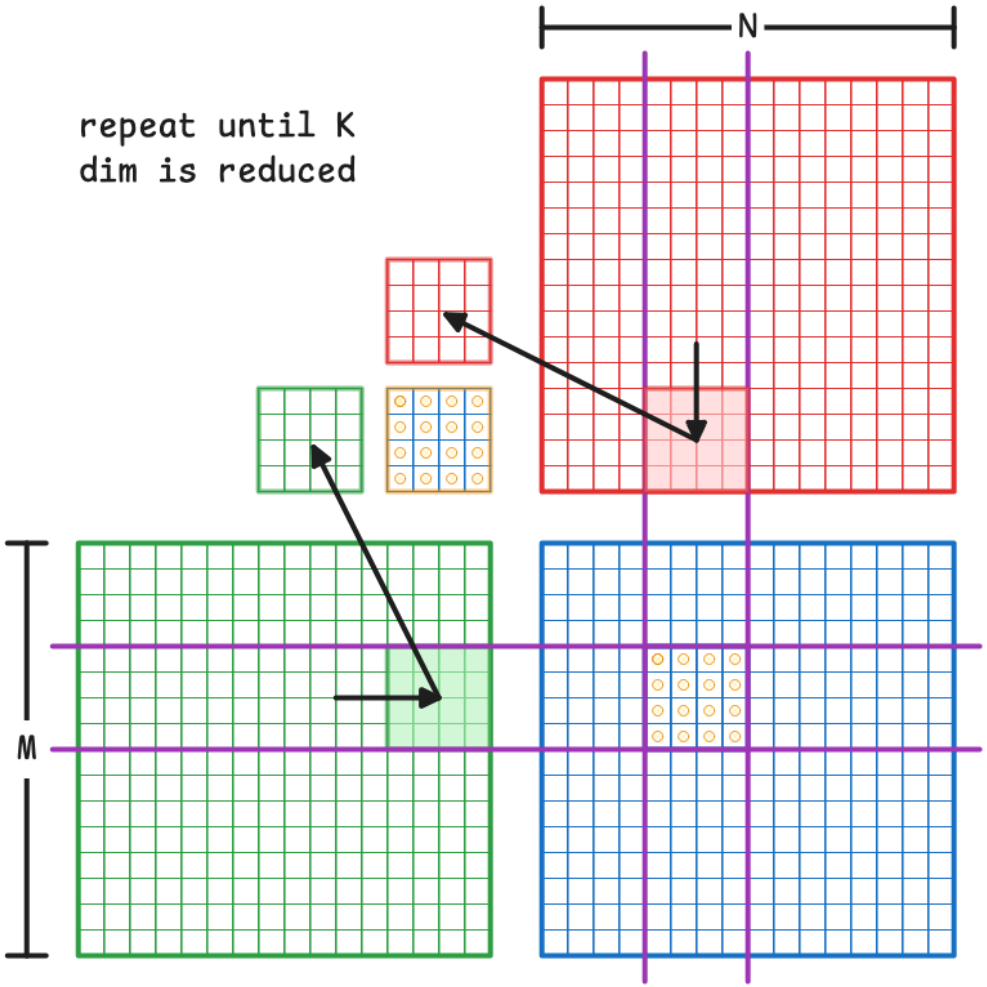
# Tiling steps



# Tiling steps



# Tiling steps



# Basic tiled SGEMM

```
__shared__ float As[BM * BK];
__shared__ float Bs[BK * BN];
float tmp = 0.0;

for (int bkIdx = 0; bkIdx < K; bkIdx += BK) {
  As[threadRow * BK + threadCol] =
      A[threadRow * K + threadCol];
  Bs[threadRow * BN + threadCol] =
      B[threadRow * N + threadCol];

  __syncthreads();
  A += BK;
  B += BK * N;

  for (int dotIdx = 0; dotIdx < BK; ++dotIdx) {
    tmp += As[threadRow * BK + dotIdx] *
           Bs[dotIdx * BN + threadCol];
  }

  __syncthreads();
}
```

**Cooperate**
Each thread loads one value from $A$ and one from $B$.

**Synchronize**
Wait until both shared tiles are complete.

**Reuse**
One thread accumulates from the cached row and column.

**Advance**
Protect the cache, move along $K$, and repeat.

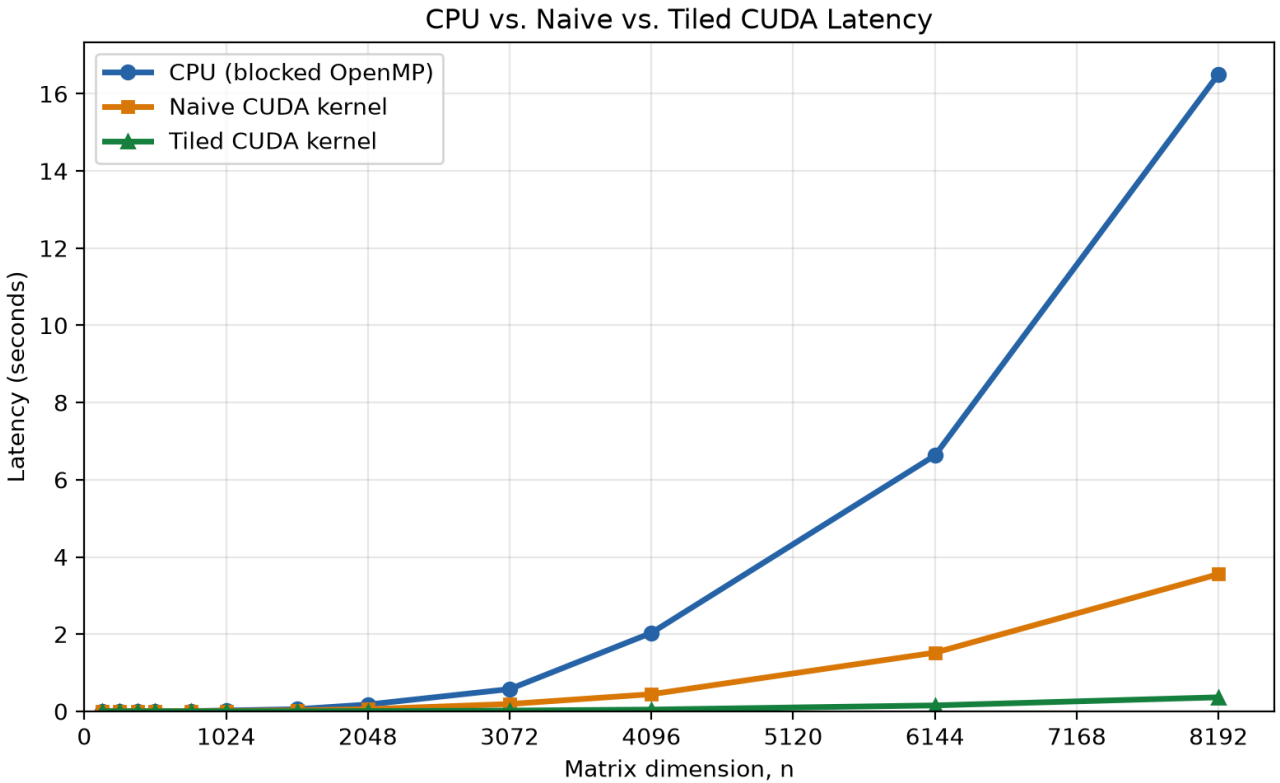
Adapted from Boehm (2022), kernel 3.

# Basic tiled SGEMM visualization

- Watch each block load small regions of $A$ and $B$ together.
- The loaded tiles are reused to update an entire tile of $C$.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

# Tiling cuts latency by another order of magnitude



# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

At 935.8 GB/s, we need 0.147s or 147ms to move all data from DRAM to SMEM.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

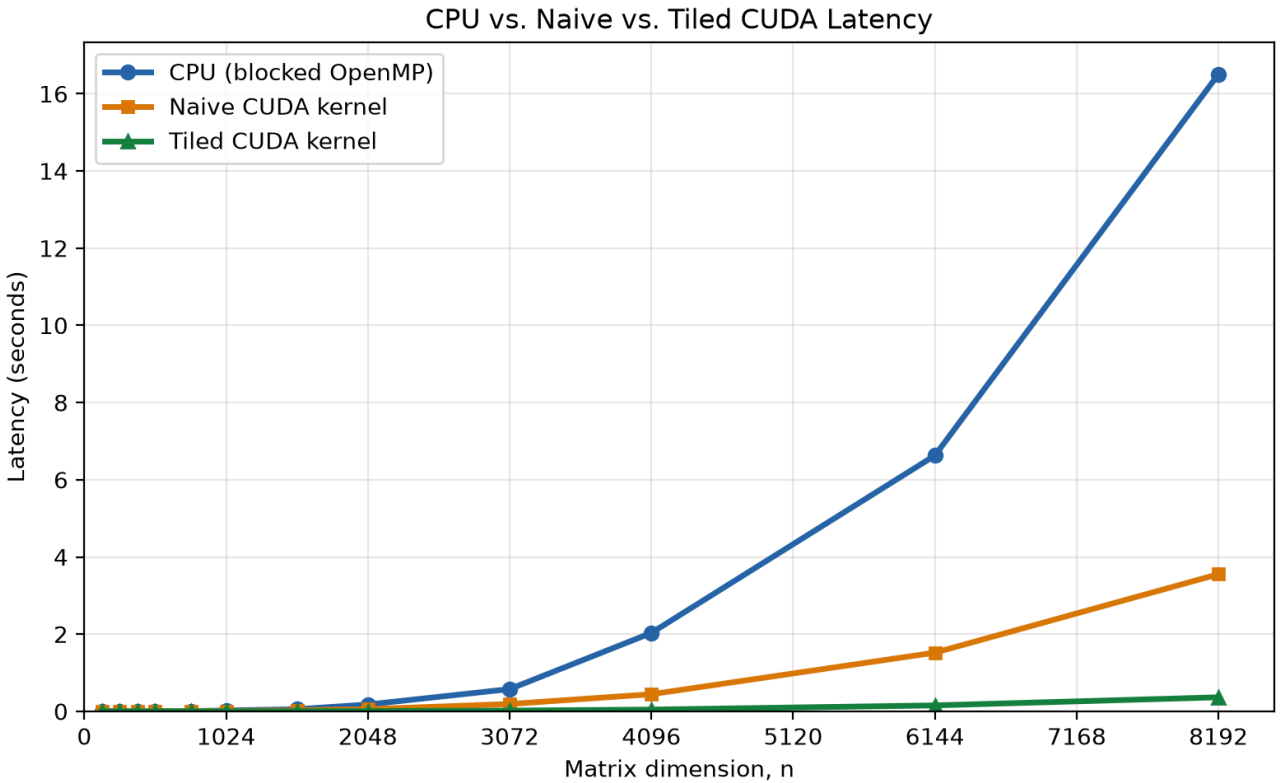
At 935.8 GB/s, we need 0.147s or 147ms to move all data from DRAM to SMEM. **40% of our latency is moving data.**

# Section Overview


1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

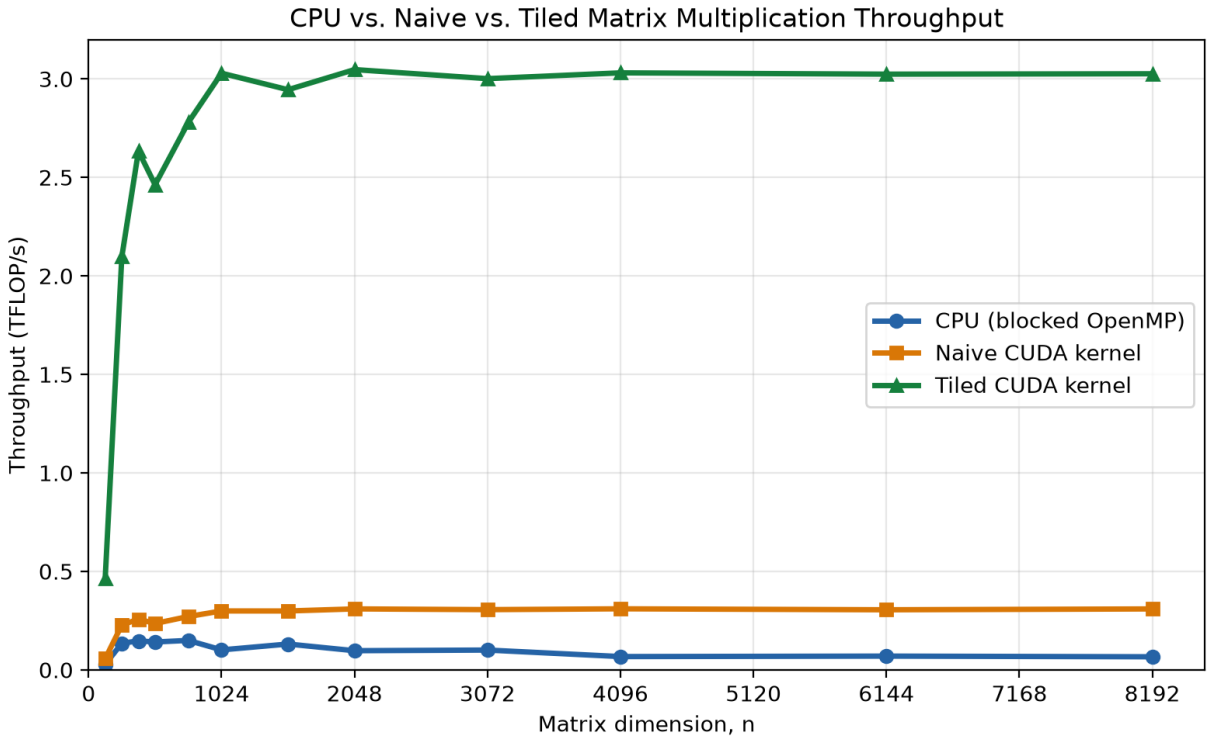
# Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.



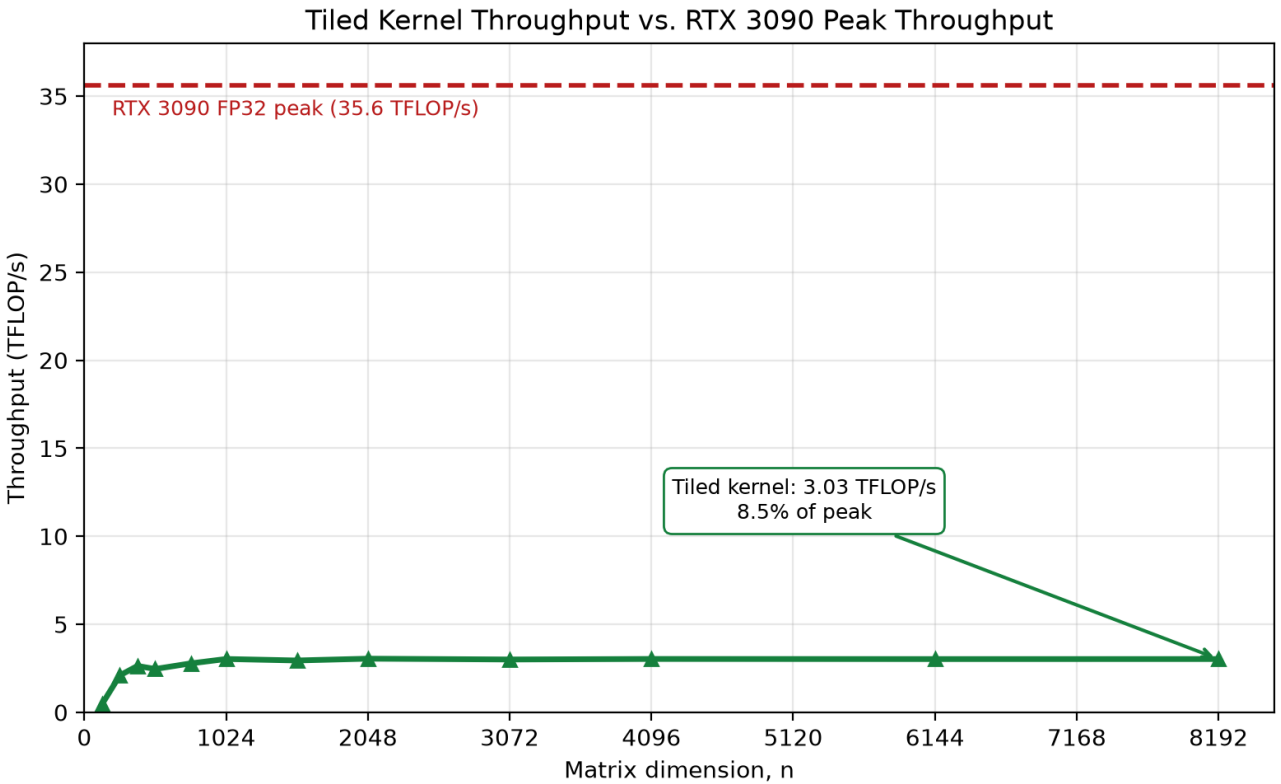
# Why latency alone isn't enough

- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.



# Why latency alone isn't enough

- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.



# Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.
- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.
- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.

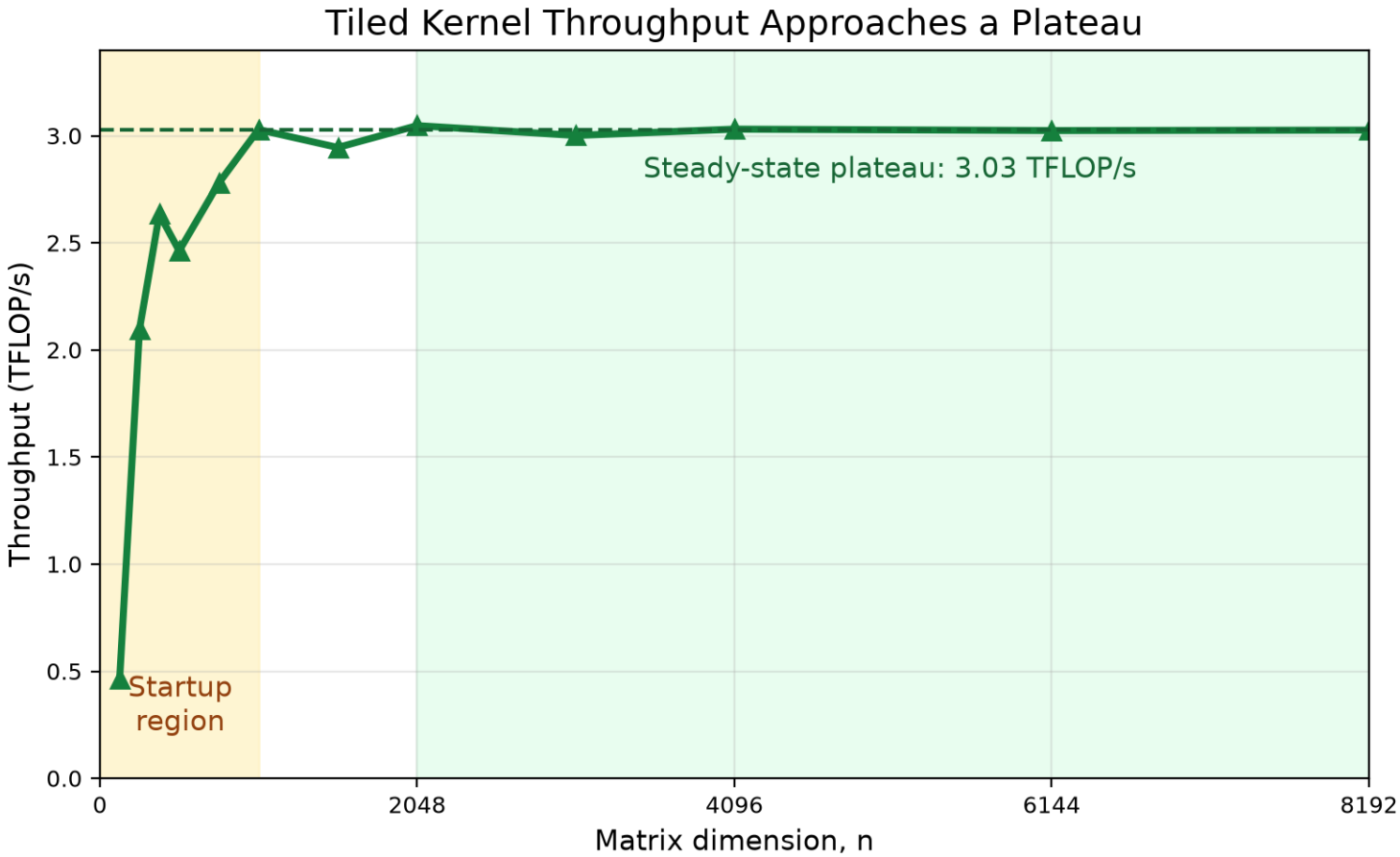
## Why throughput

Throughput tells us

1. A general performance metric, no matter the problem size
2. How well we are utilizing hardware

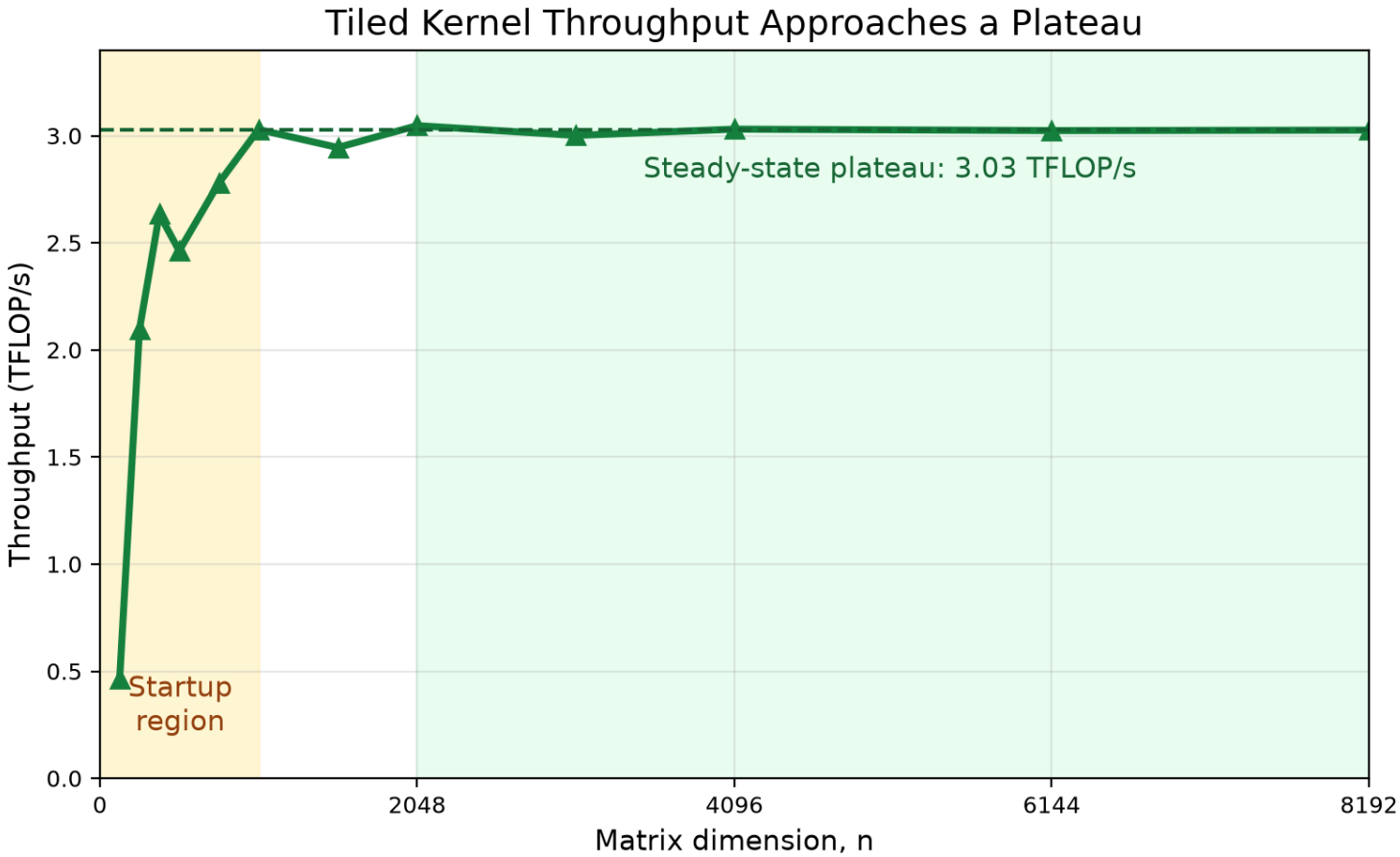
# Why throughput still depends on problem size

Normalizing by useful work makes comparisons fairer, not size-independent.



# Why throughput still depends on problem size

Normalizing by useful work makes comparisons fairer, not size-independent.




## Measurement rule

Measure in the plateau region, and report the problem size with the result.

# Throughput as a performance measure

## Throughput

**Throughput** is the total useful work (FLOPs) done per unit time. *We previously discussed theoretical peak throughput of hardware.*

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

## Example

Peak throughput of our RTX 3090 is 35.6 TFLOP/s. We observed 360ms or 0.36s of latency for $n = 8192$, where the total FLOPs required is 1.10 TFLOPs. Thus, we have an observed throughput of

$$\frac{1.10\text{TFLOPs}}{0.36\text{s}} \approx 3.05\text{TFLOP/s}.$$

Roughly 8.5% of peak throughput.

# Arithmetic Intensity

## Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

## Example

For $n, k, m = 8192$, our tiled kernel moved a total of 137.7 GB of data from DRAM. With a total work count of 1.10 TFLOP, its arithmetic intensity is

$$\text{AI} = \frac{1.10 \times 10^{12} \text{ FLOP}}{137.7 \times 10^{9} \text{ bytes}} \approx \boxed{8.0 \text{ FLOP/byte}}.$$

# Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

# Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.6 GB/s of bandwidth, our **memory roof** is

$$935.8 \text{ GB/s} \times 8.0 \text{ FLOP/byte} \approx 7.49 \text{ TFLOP/s}.$$

# Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.6 GB/s of bandwidth, our **memory roof** is

$$935.8 \text{ GB/s} \times 8.0 \text{ FLOP/byte} \approx 7.49 \text{ TFLOP/s}.$$

This is much lower than the peak throughput of 35.6 TFLOP/s.

# Two Key Constraints of Hardware

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.6 GB/s of bandwidth, our **memory roof** is

$$935.8 \text{ GB/s} \times 8.0 \text{ FLOP/byte} \approx 7.49 \text{ TFLOP/s}.$$

This is much lower than the peak throughput of 35.6 TFLOP/s. **No matter how we optimize this kernel, without increasing AI, we will never achieve greater than 7.49 TFLOP/s**.

# Hypothetical Kernel

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s.

# Hypothetical Kernel

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s. Our upper bound throughput is now

$$935.8 \text{ GB/s} \times 16.0 \text{ FLOP/byte} \approx 15.0 \text{ TFLOP/s}.$$

# Hypothetical Kernel

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

## Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s. Our upper bound throughput is now

$$935.8 \text{ GB/s} \times 16.0 \text{ FLOP/byte} \approx 15.0 \text{ TFLOP/s}.$$

Since 15.0 TFLOP/s $<$ 35.6 TFLOP/s, this kernel will still be memory bound.

# Hypothetical Kernels

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

| AI (FLOP/byte) | Upper bound throughput (TFLOP/s) |
|---|---|
| 8 | 7.49 |
| 16 | 14.97 |
| 32 | 29.95 |
| 36 | 33.69 |
| 38 | 35.56 |
| 40 | 37.43 |

# Hypothetical Kernels

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

| AI (FLOP/byte) | Upper bound throughput (TFLOP/s) |
|---|---|
| 8 | 7.49 |
| 16 | 14.97 |
| 32 | 29.95 |
| 36 | 33.69 |
| 38 | 35.56 |
| 40 | 37.43 |

Wait! Our peak throughput is 35.6 TFLOPs! At AI $\geq$ 38, we are **compute bound**. Our data can finally keep up with the CUDA Cores!

# Ridge Point

Our hardware is characterized by

1. Peak throughput: 35.6 TFLOP/s
2. DRAM bandwidth: 935.8 GB/s

AI = 38 FLOP/byte is known as the **ridge point** *RP*, and can be found by solving:

$$\text{Data roof} = \text{Peak throughput}$$

$$\text{DRAM Bandwidth} \times \text{RP} = \text{Peak throughput}$$

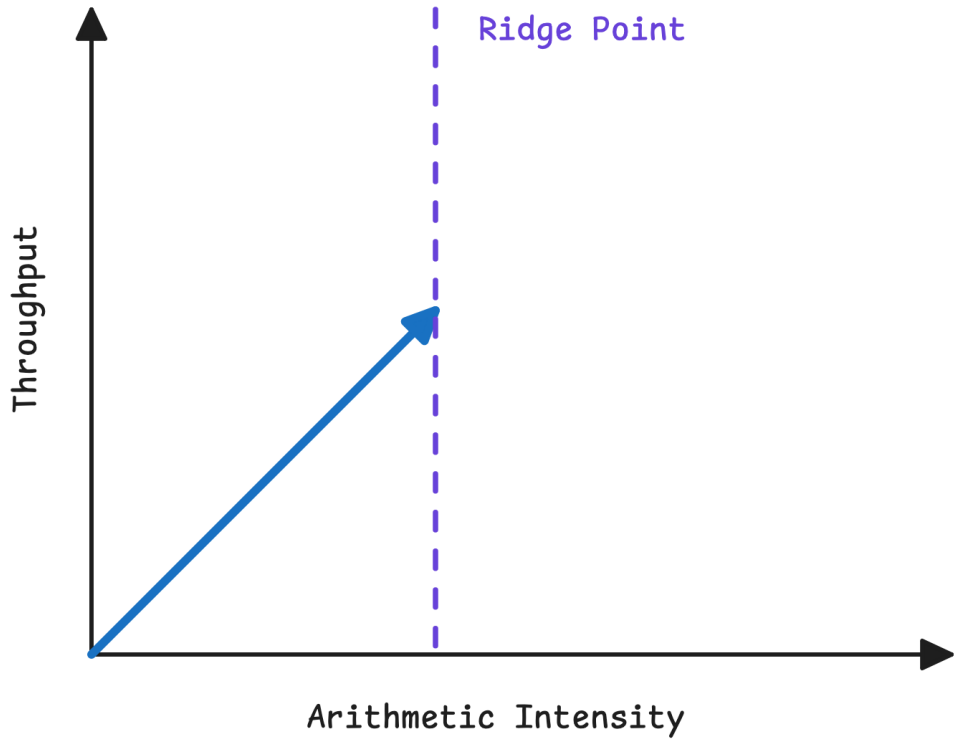
$$\text{RP} = \frac{\text{Peak throughput}}{\text{DRAM Bandwidth}}$$

$$\text{RP} = \frac{35.6 \text{ TFLOP/s}}{935.8 \text{ GB/s}} \approx 38.0 \text{ FLOP/s}$$

# Roofline Model

## Roofline Model

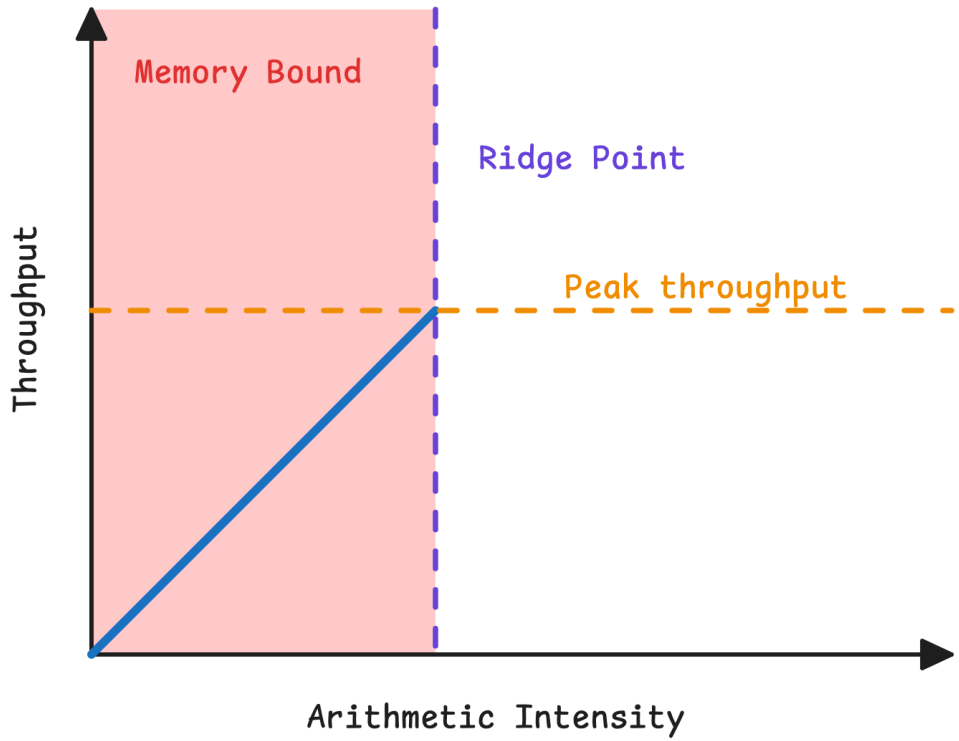
For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI.



# Roofline Model

## Roofline Model

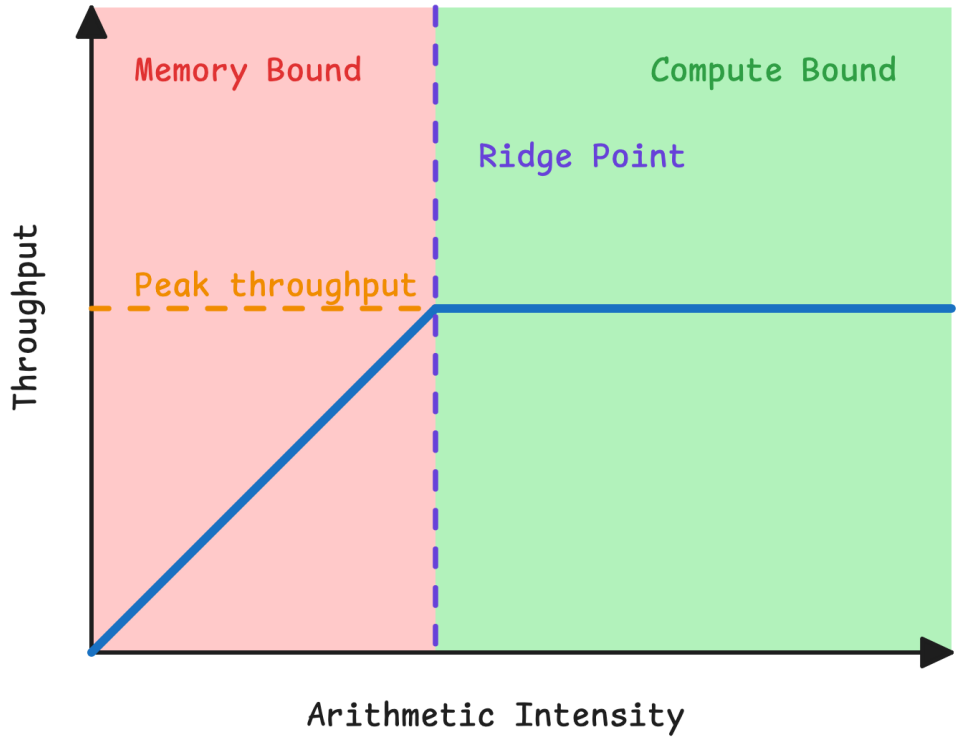
For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput.



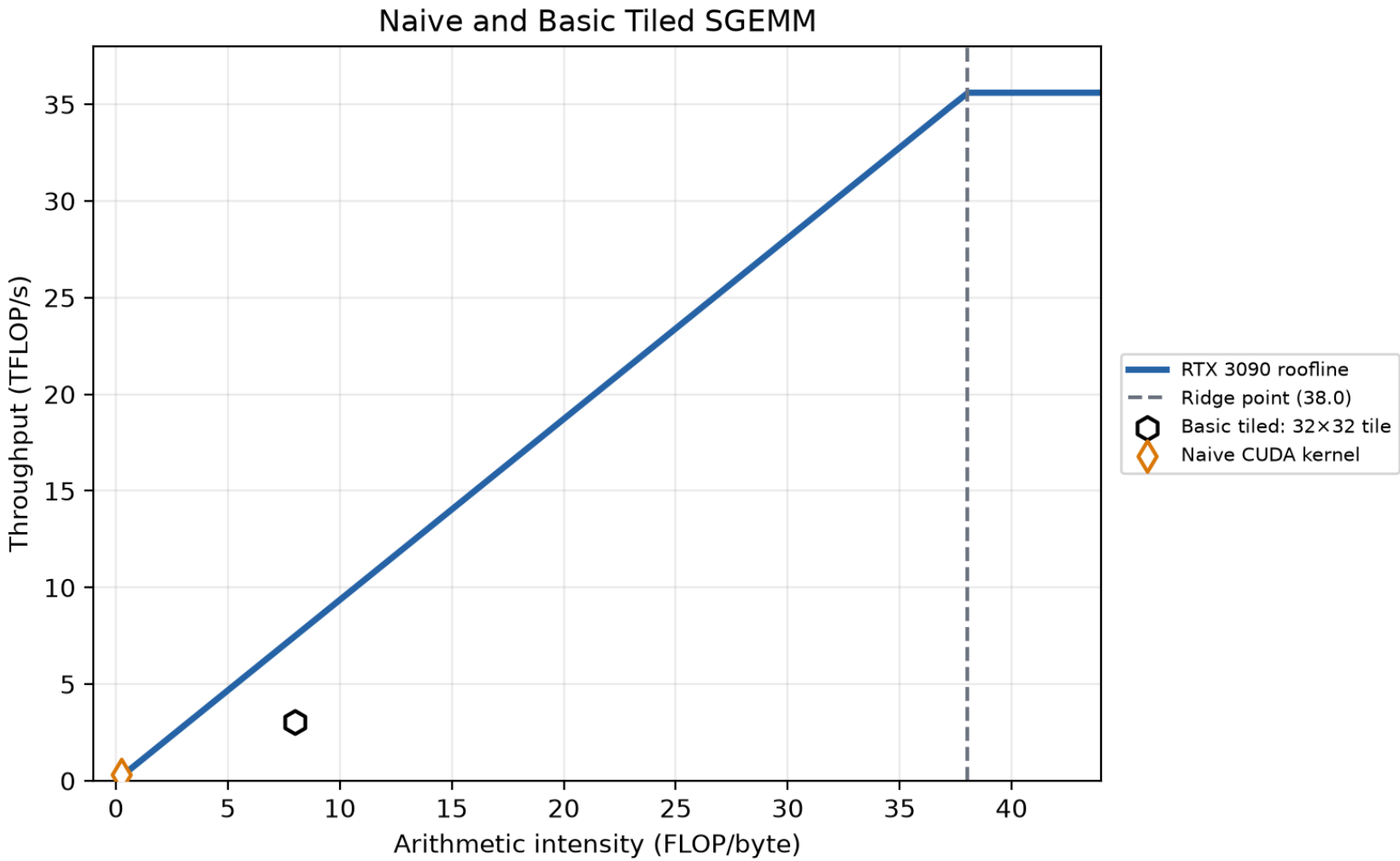
# Roofline Model

## Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. Past that, we are **compute bound**, increasing AI will not raise the upper bound of throughput.



# Roofline Model of our RTX 3090

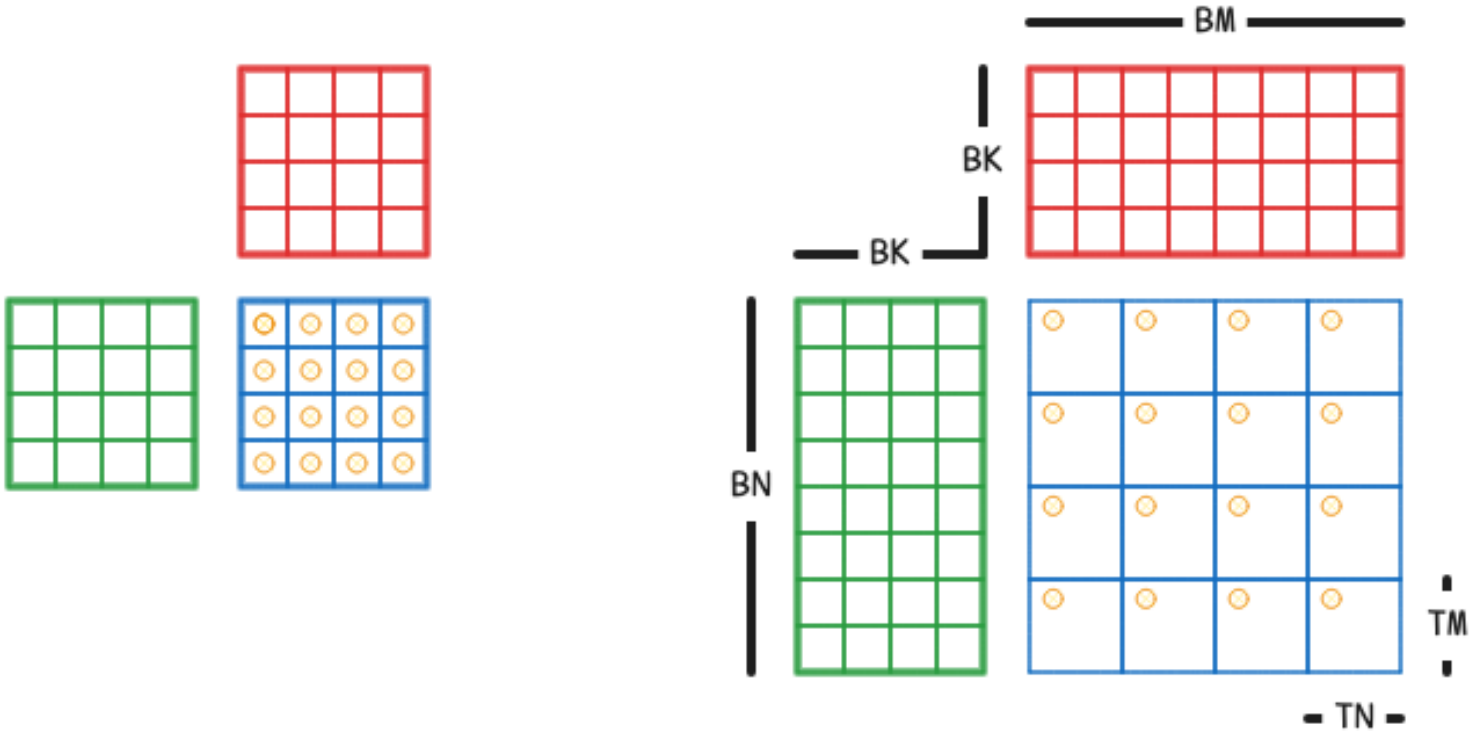


# Section Overview

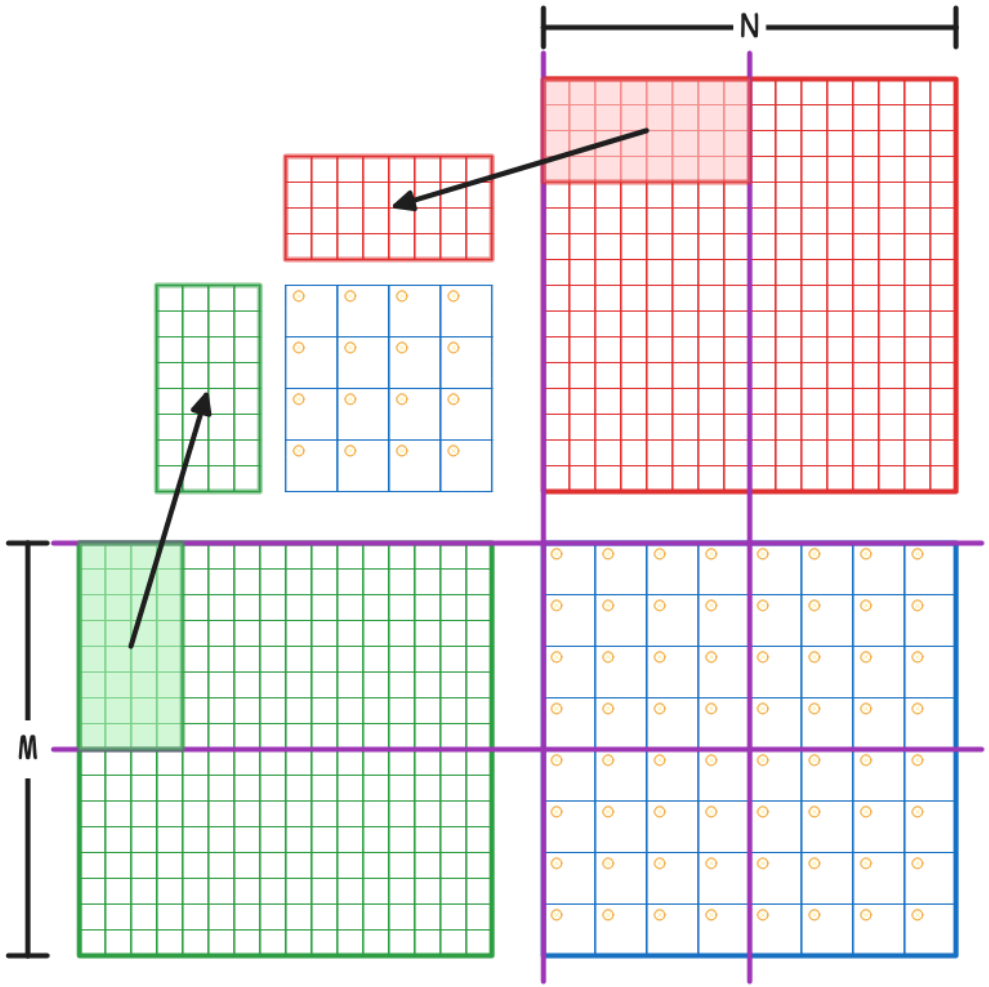

1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

# Thread coarsening: Increasing AI

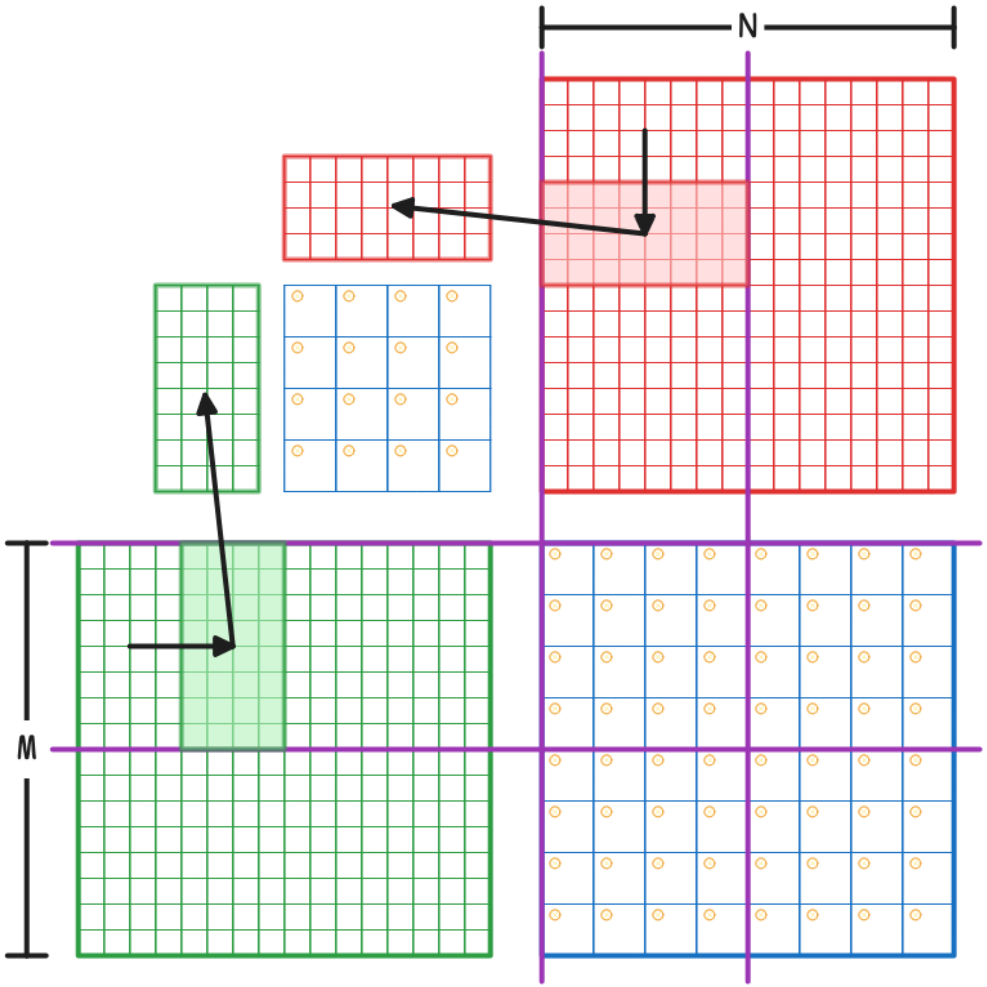
- Basic tiling assigns 1 element of the output to each thread.
- We can *coarsen* each thread, and assign a $TM \times TN$ portion of the output to each thread.
- We load larger chunks into SMEM, increasing data reuse and AI.
- One loaded $A$ value can update several columns; one loaded $B$ value can update several rows.



# Coarsened SGEMM visualization



# Coarsened SGEMM visualization



# Coarsening factors are a tradeoff

## Two levels of tiling

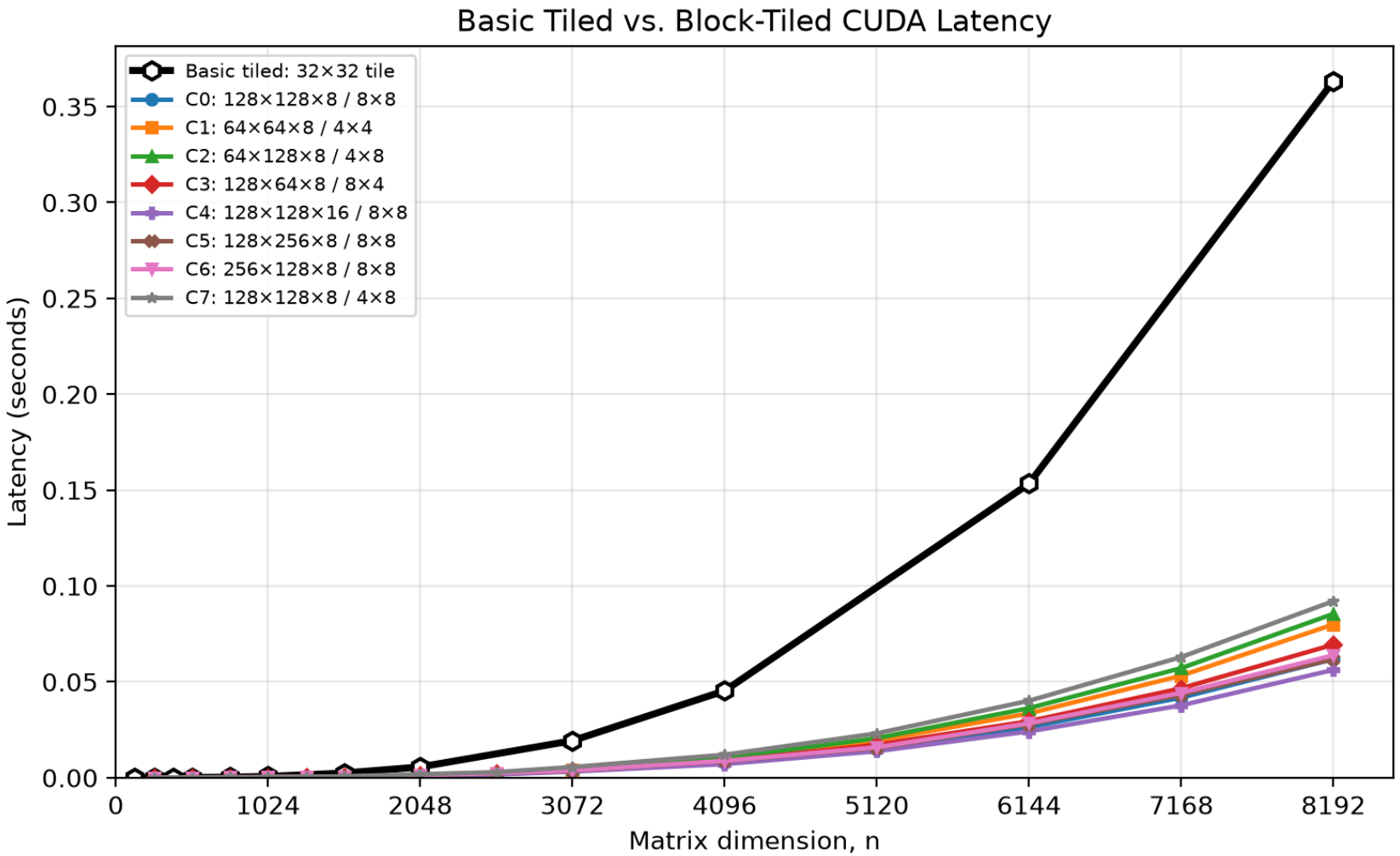
- A block computes a $BM \times BN$ output tile while advancing through $K$ in steps of $BK$.
- Each thread computes a $TM \times TN$ micro-tile in registers.
- Thread count: $\frac{BM}{TM}\frac{BN}{TN}$.

## Why not keep increasing them?

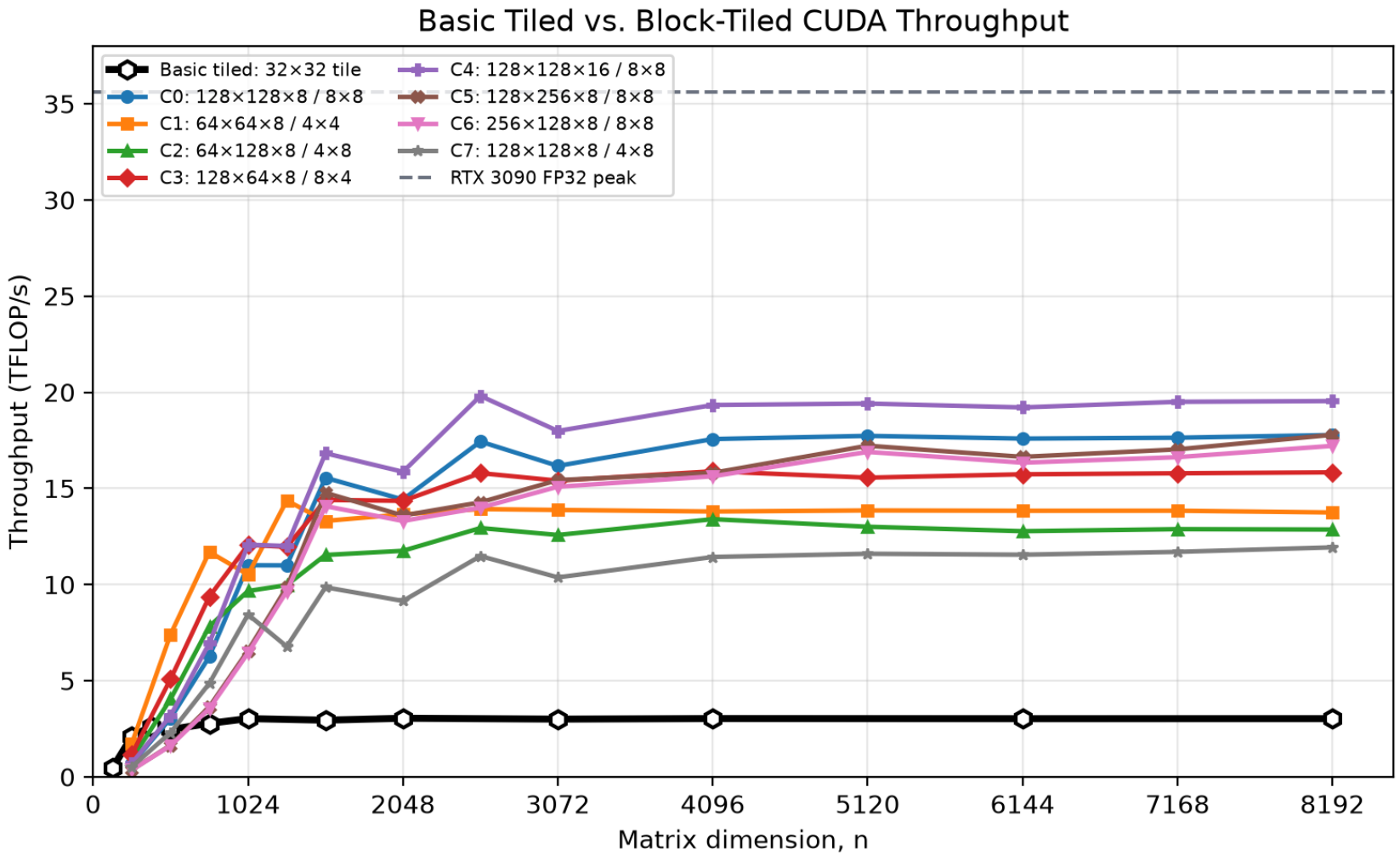
Larger tiles improve reuse and arithmetic intensity, but consume more registers and shared memory. Eventually occupancy falls and extra parallelism or reuse no longer helps.

| Config. | $BM \times BN \times BK$ | $TM \times TN$ | AI | Throughput |
|---|---|---|---|---|
| C1 | $64 \times 64 \times 8$ | $4 \times 4$ | 15.9 | 13.75 TFLOP/s |
| C4 | $128 \times 128 \times 16$ | $8 \times 8$ | 31.5 | **19.54 TFLOP/s** |
| C5 | $128 \times 256 \times 8$ | $8 \times 8$ | 41.8 | 17.79 TFLOP/s |

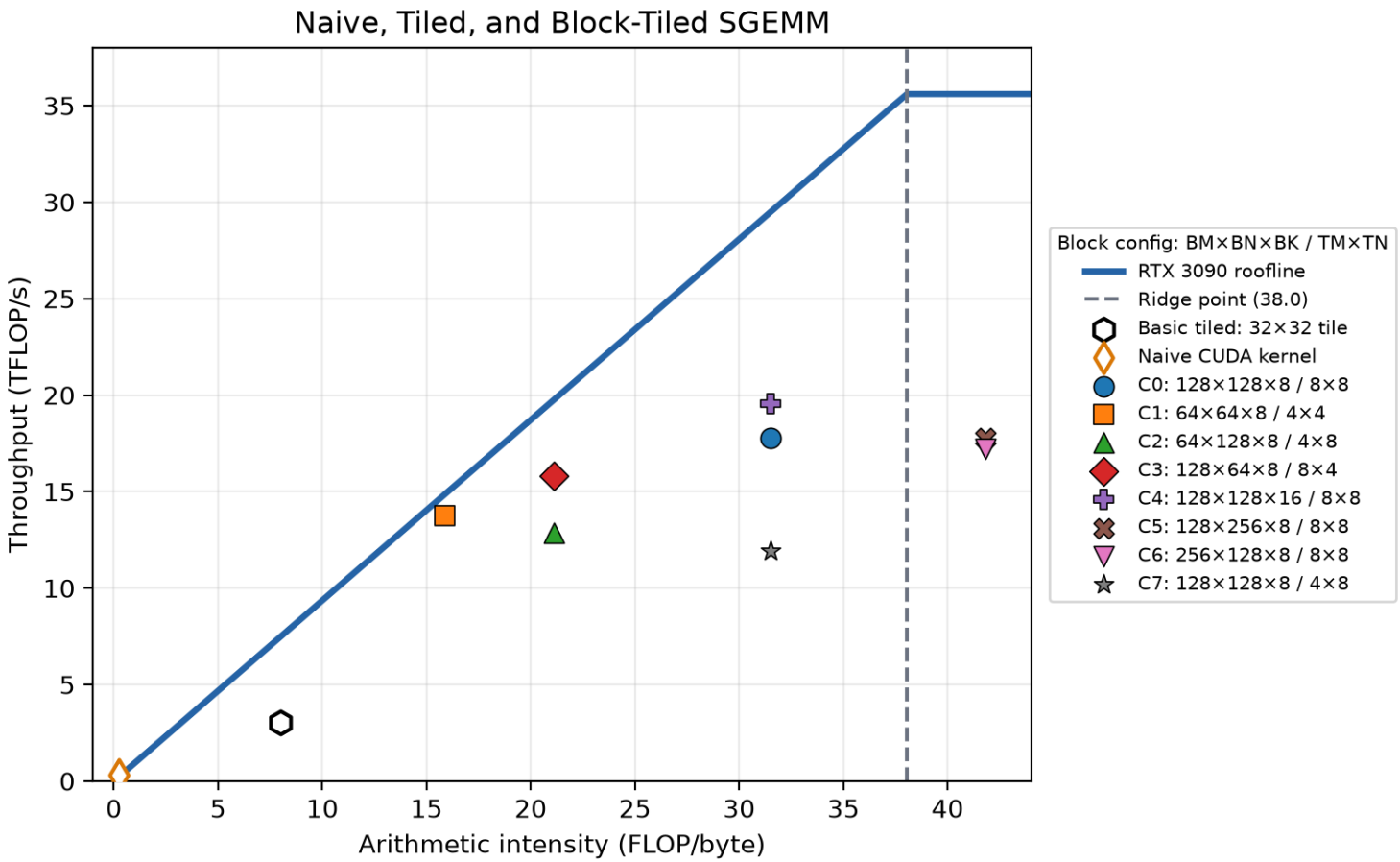
# Basic tiled vs. block-tiled latency



# Block tiling raises useful throughput



# Tiling configurations on the roofline




Reminder that the Roofline Model only tells us the upper bound!

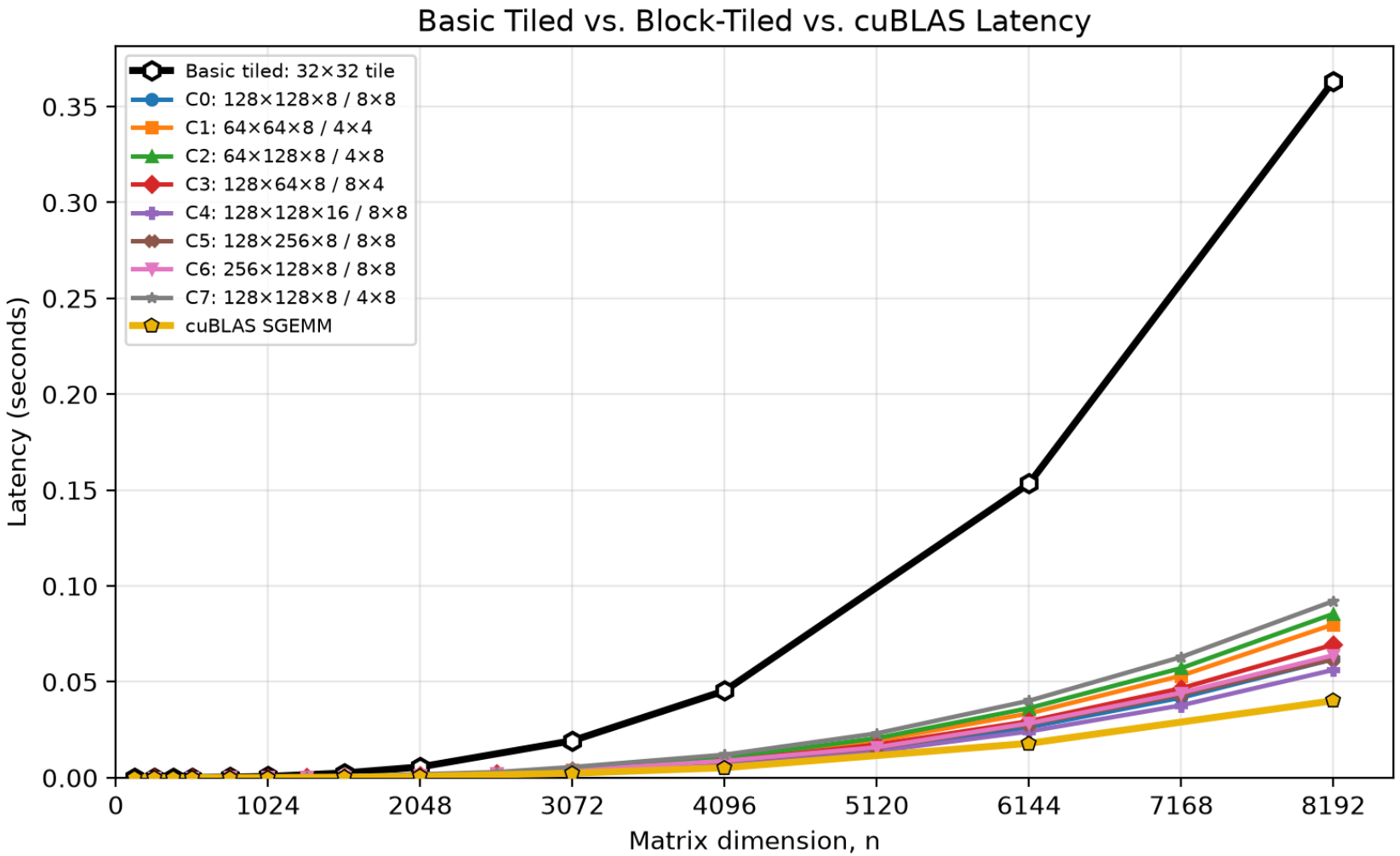
# We have only scratched the surface: cuBLAS

- cuBLAS is NVIDIA's architecture-tuned linear algebra library and the practical performance target for our SGEMM kernels.
- At $n = 8192$, cuBLAS reaches 27.38 TFLOP/s, while our fastest configuration (C4) reaches 19.54 TFLOP/s, or 71.4% of cuBLAS's throughput.
- cuBLAS completes the multiplication in 40.2 ms versus 56.3 ms for C4, about 1.40× faster.

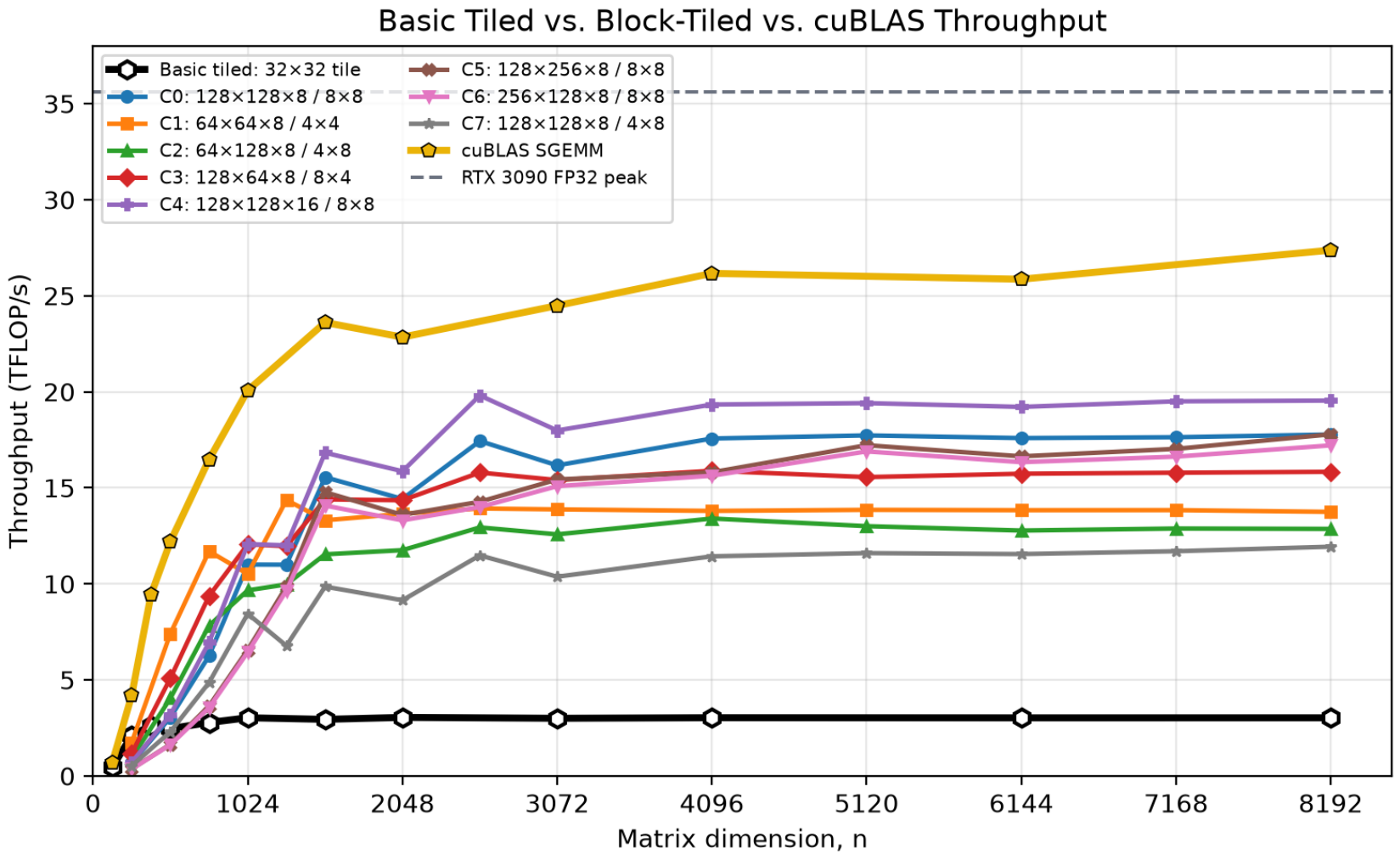
**The remaining gap**

Production kernels combine deeper register and warp tiling, vectorized memory access, pipelined loads, and architecture-specific instruction scheduling.

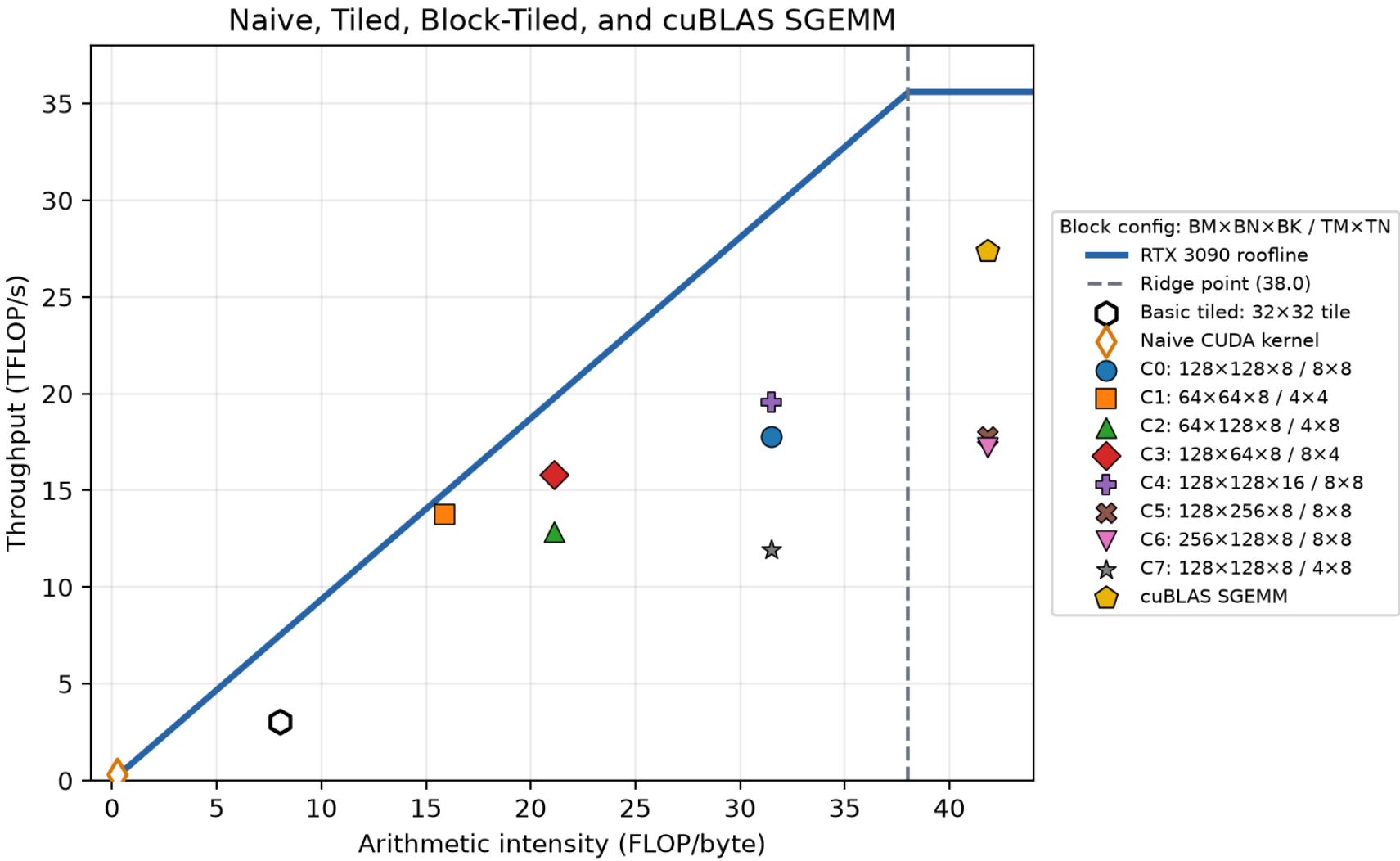
# Basic tiled vs. block-tiled latency vs. cuBLAS



# cuBLAS remains the performance target

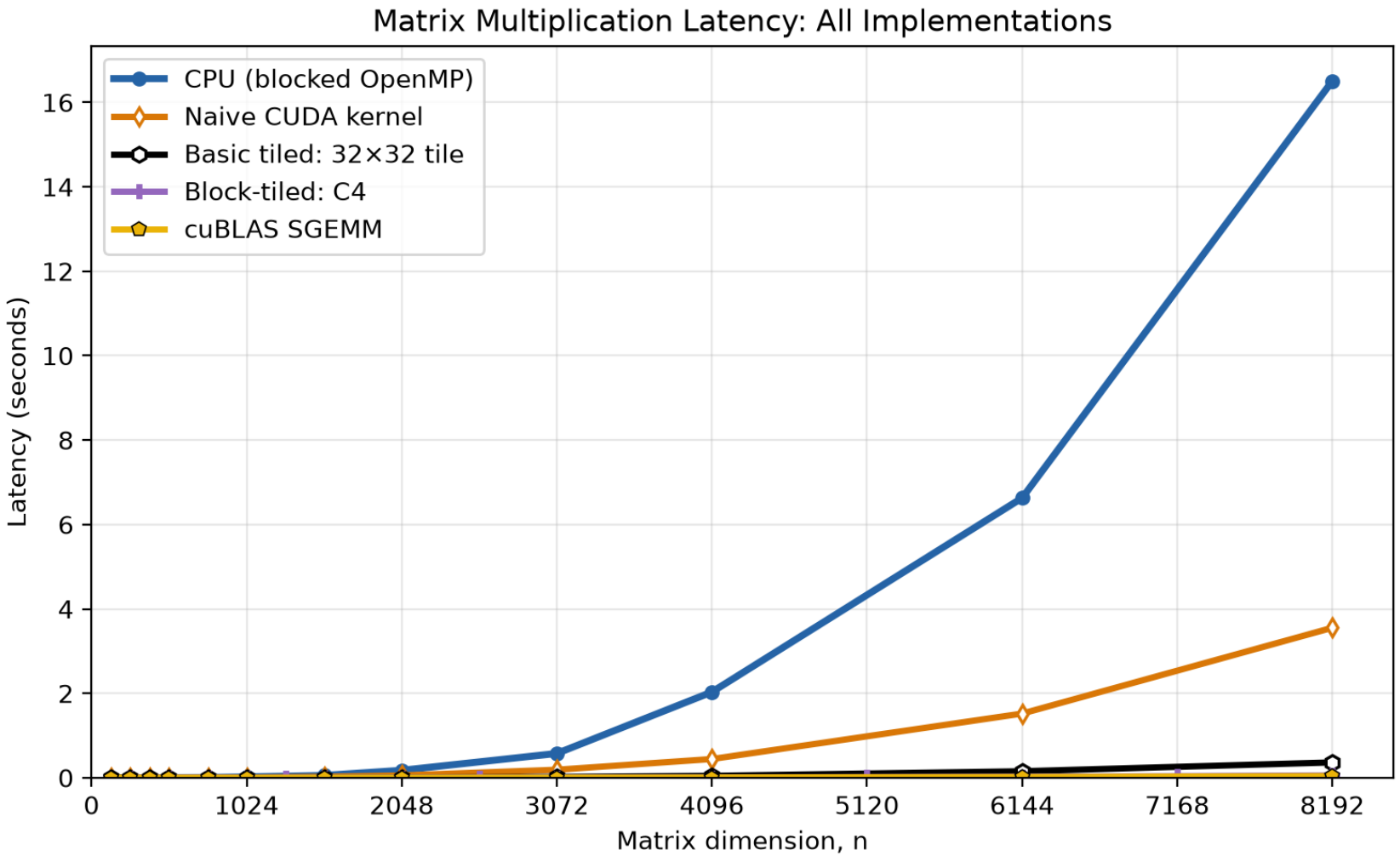


# cuBLAS on the roofline




cuBLAS arithmetic intensity is estimated from the profiled $256 \times 128$ output-tile shape.

# We've come a long way



# Table of Contents


1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

# Specialized kernels

A general SGEMM kernel must handle many shapes and layouts. Real models let us trade some of that flexibility for more performance.

## What can be specialized?

- Fixed tensor shapes and memory layouts
- Reduced precision and Tensor Core tile sizes
- Structured sparsity or quantized weights
- Fused bias, activation, and normalization

## Why does it help?

- Less indexing and control-flow overhead
- Fewer kernel launches and intermediate writes
- Better use of registers and specialized hardware
- Autotuning can choose a kernel for each layer

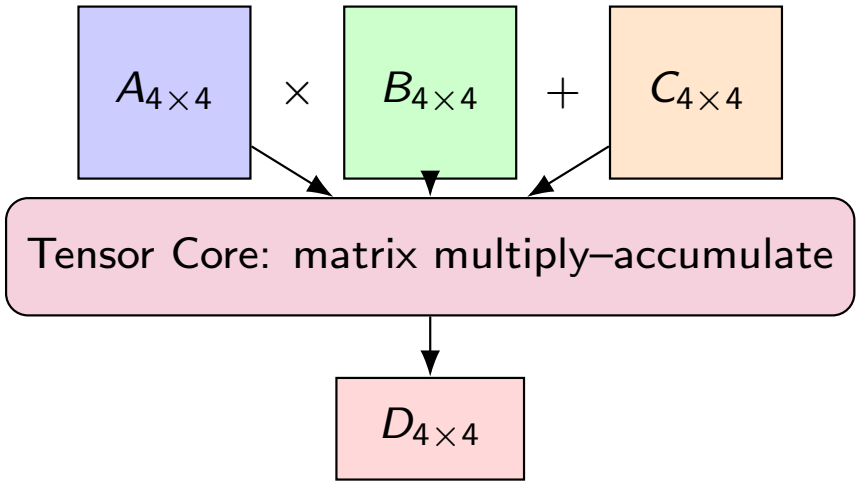
## Example: fuse the epilogue

Compute $Y = \mathrm{ReLU}(XW + b)$ in one kernel instead of writing $XW$ to global memory and reading it back for bias and activation.

# Continuing GPU evolution

## 2017: Volta Tensor Cores

- Tensor Cores perform mixed-precision matrix multiply–accumulate: $D = AB + C$.
- The original Volta unit multiplies FP16 inputs and accumulates in FP16 or FP32.
- Matrix multiplication becomes an explicit hardware primitive, alongside general-purpose CUDA cores.



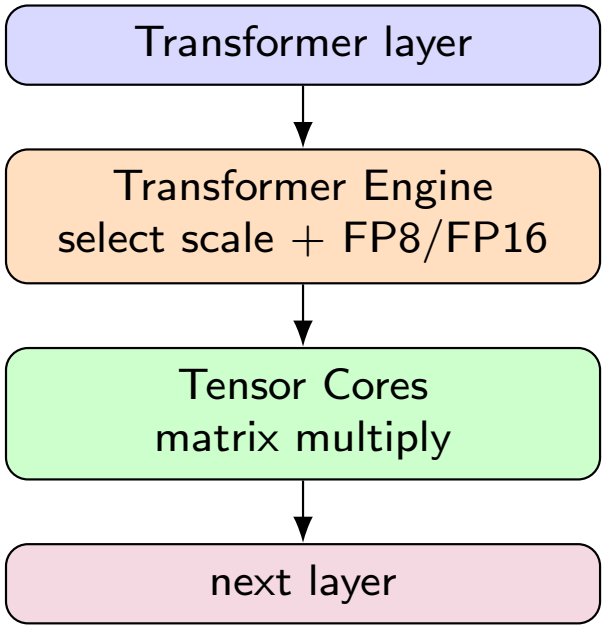

Operation shape shown for first-generation Volta Tensor Cores. Source:

NVIDIA, *Volta Architecture Whitepaper*, 2017.

# Continuing GPU evolution

**2020 onward: specialize around AI dataflow**

- Ampere (2020) adds TF32 and structured sparsity support.
- Hopper (2022) adds FP8 Tensor Cores and a Transformer Engine that dynamically chooses FP8 or FP16 per layer.
- Blackwell (2024) adds more low-precision formats and a second-generation Transformer Engine. This is more than incremental scaling.




Source: NVIDIA, *Hopper Architecture In-Depth* (2022).

# Continuing GPU evolution

## 2026: Vera Rubin—optimization moves to the rack

- The target is no longer an isolated GPU, but a complete rack-scale computing platform.
- Vera CPUs, Rubin GPUs, memory, NVLink, and networking are designed together to run one distributed AI workload.
- A high-bandwidth scale-up fabric lets many accelerators share model parameters and intermediate results.
- Performance now depends on communication, power, cooling, and software orchestration as well as matrix throughput.

**Unit of optimization**

Kernel

↓

GPU

↓

Server

↓

**Rack-scale system**

Source: NVIDIA, *Vera Rubin Architecture*, announced roadmap.

# Alternative accelerator architectures

**Different vendor, or different architecture?**

## AMD Instinct: another GPU ecosystem

AMD's datacenter GPUs use many programmable compute units, matrix instructions, and high-bandwidth memory. ROCm provides the software stack used to program them.

**Strength:** retains GPU flexibility while competing with CUDA.
**Tradeoff:** performance depends heavily on software and library support, not only the silicon.

## Google TPU: systolic arrays

Data moves rhythmically through a two-dimensional grid of multiply–accumulate units. Values are reused as they pass between neighboring units instead of repeatedly returning to memory.

**Strength:** efficient, predictable dense tensor operations.
**Tradeoff:** less flexible for irregular workloads.

Sources: AMD, *Instinct Accelerators*; Jouppi et al. (2017).

# Alternative accelerator architectures

**Other designs change the scale or location of computation.**

## Cerebras: wafer-scale integration

Many compute cores, distributed SRAM, and an on-wafer fabric are placed on one large processor. More communication stays on the wafer instead of crossing between accelerator packages.

**Strength:** memory capacity and bandwidth close to compute.
**Tradeoff:** difficult manufacturing, cooling, and mapping.

## NPUs: AI inside the device

NPUs sit beside CPUs and GPUs in phones, laptops, and embedded systems. They accelerate supported quantized tensor operations under strict power and thermal limits.

**Strength:** private, low-latency inference without the cloud.
**Tradeoff:** limited memory and operator support.

Sources: Cerebras, *Wafer-Scale Engine*; Sze et al. (2017).

# Alternative accelerator architectures

## Model ASIC: Etched Sohu

The hardware and dataflow target transformer inference rather than arbitrary kernels. Removing general-purpose machinery can devote more silicon and energy to the intended model family.

**Strength:** potentially much higher throughput per watt.
**Tradeoff:** model changes can make fixed hardware obsolete.

## The design spectrum

GPU: programmable

↓

TPU/NPU: operator-focused

↓

ASIC: model-focused

More specialization ⇒ less flexibility

## There is no universally best architecture

Specialization can improve performance per watt, but reduces flexibility. The right balance depends on workload stability, scale, power, and the cost of moving data.

Source: Etched (2024), "Introducing Sohu."

# Table of Contents


1. Neural Networks and the Matrix Multiplication
2. Why GPUs? What's wrong with CPUs?
3. From graphics pipeline to AI accelerator
4. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - Memory Boundedness, Basic Tiling, and Analysis
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening
5. Future discussions
6. Conclusion and References

# Conclusion

1. **AI workloads expose parallel linear algebra.** Neural-network layers repeatedly reduce to matrix and tensor operations.
2. **GPUs already had the right foundation.** Graphics demanded parallel throughput and high memory bandwidth; unified shaders and CUDA made that hardware programmable.
3. **Data movement determines real performance.** Coalescing, shared-memory tiling, and register reuse move GEMM toward the compute limit.
4. **AI is now driving specialization.** Tensor Cores, lower precision, and new accelerators trade flexibility for throughput and efficiency.

**Main takeaway**

Graphics and AI both reward massive parallelism. The GPU succeeded by evolving from a graphics pipeline into a programmable matrix engine.

# References: GPU evolution


- I. Buck et al. (2004), "Brook for GPUs: Stream Computing on Graphics Hardware."
- E. Lindholm et al. (2008), "NVIDIA Tesla: A Unified Graphics and Computing Architecture."
- NVIDIA (2009), *Fermi Compute Architecture Whitepaper*.
- NVIDIA (2017), *NVIDIA Tesla V100 GPU Architecture*.
- NVIDIA (2020), "NVIDIA Ampere Architecture In-Depth."
- NVIDIA (2022), "NVIDIA Hopper Architecture In-Depth."
- NVIDIA (2024), "NVIDIA Blackwell Architecture."
- NVIDIA, "NVIDIA Rubin Architecture."

# References: CUDA and GEMM


- S. Boehm (2022), "How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance."
- S. Williams, A. Waterman, and D. Patterson (2009), "Roofline: An Insightful Visual Performance Model for Multicore Architectures."
- NVIDIA, *CUDA C++ Programming Guide*.
- NVIDIA, "GeForce RTX 3090 Family Specifications."
- NVIDIA, "CUDA Refresher: The CUDA Programming Model."
- NVIDIA, *cuBLAS Library Documentation*.
- Modal, "CUDA Programming Model," *GPU Glossary*.
- F. Sanglard (2020), "The CUDA Architecture."

# References: alternative accelerators


- AMD, "AMD Instinct Accelerators."
- N. P. Jouppi et al. (2017), "In-Datacenter Performance Analysis of a Tensor Processing Unit."
- V. Sze et al. (2017), "Efficient Processing of Deep Neural Networks: A Tutorial and Survey."
- Cerebras, "The Wafer-Scale Engine."
- Etched (2024), "Introducing Sohu: The World's First Transformer ASIC."


Questions?